symGCAD — corpus benchmark

generated 2026-10-12

All 331 distinct problems of the GCAD corpus: exact generic CAD in Rust on Symbolica, against the original GCAD by Vitaly Magerya

1 Benchmark results

STRICT CONTRACT
303 / 331
independently verified · fixed 124/124 · symbolic 97/124 · core 82/83.

OPT-IN PROJECTIVE
+1
test_gluza_ex33/solve/3 completes only under `projective_sections = true` (§1.2); 2 of 2 opt-in rows verified. **Not** counted in the strict total.

NATIVE GCAD, 300 S/CASE
278 / 331
groups completed · 53 timeout · 0 out of memory (2 sign cases hit `std::bad_alloc` at 8 GiB and were re-run at 24 GiB, where they timed out); cases 486/579 complete.

GEOMETRIC MEAN RATIO
0.056
symGCAD solve / native time on the 278 groups both complete (17.7× faster; dominated by native Python start-up, against native internal time 0.94); 274 of 278 faster, 2 more than 2× slower. Where native needs ≥ 1 s (41 groups): 0.061, or 0.086 against native internal time.

ONLY SYMGCAD COMPLETES
25
groups verified by symGCAD where native GCAD timed out or ran out of memory. Summed times on the common set: 138.9 s vs 747.5 s.

What is compared. Each row is one distinct original problem (aliases merged; split polynomials kept). **symGCAD** solves the full problem in one fresh process with its recipe (engine, ordering policy, options from the 2026-10-10 audit, `benchmark/campaign/symgcad_inputs/`), then `symgcad verify` re-checks the exported proof in a second fresh process. Only a verified `complete_generic` result counts. **Native** is the unmodified GCAD (`fb6ebe3`, FLINT 3.6) through the unmodified bridge `reference/adapter.py`: each sign case of the split (`F > 0` and `F < 0` added as constraints, as in the earlier parity campaign) is solved in its own process, capped at 300 s and 8 GiB (the two sign cases that hit `std::bad_alloc` at 8 GiB were repeated with 24 GiB and then timed out at 300 s); the native time is the sum over sign cases of GNU `time`'s elapsed wall time (10 ms resolution) and includes Python start-up and SymPy parsing. The **ratio** is symGCAD solve / native time; green < 1 (symGCAD faster), yellow 1–2, red > 2. Most corpus problems are tiny, and there the native time is dominated by Python start-up: against native internal time alone (no start-up), the geometric mean over all 278 groups is 0.94, i.e. about parity, because symGCAD's own process time includes its start-up and export. On the 41 groups where native GCAD needs at least 1 s of internal time, the geometric mean ratio is 0.061 against native process time and 0.086 against native internal time (11.6× faster without any start-up cost on the native side).

Columns. Order is the variable order of the exported result (projection eliminates from the right); **H** marks the homogeneous anchor fixed to 1 before solving. **Method** is the engine and ordering policy (`mods` = GCAD's heuristic, `given` = recipe order, `mcts` = bounded search inside the timed solve). **Cells** are verified raw cells (no merging; GCAD merges, so cell counts are not comparable); † marks the 53 groups whose count differs from the 2026-10-10 audit (both verified): for 43 of them the exported order differs; for 10 the recipe and order are those of the audit and only the solver code changed. Times are wall seconds on one pinned CPU of an AMD EPYC 9754, cold: a fresh process, run directory and `TMPDIR` per run and no persistent checkpoint writes (they are opt-in, K2; the runner sets a fresh checkpoint directory only for an MCTS search, and none of the timed rows runs one). **Solve s** and **Verify s** are measured around each process (start-up and the export of `result.json` included) by a worker process of its own per group, pinned to that group's CPU, so no other group's bookkeeping enters a timing; each is the median of 3 cold runs, every one of which verified the same cell count. On the smallest problems (about 0.05 s) the time is mostly process start-up and the export, whose copy of the worker's output on this shared ZFS file system can wait for a file-system sync (up to about 0.5 s with 0.01 s of CPU); the repeats damp this. **Native s** is GNU `time`'s elapsed time. Verified rows are sorted by solve time, then the incomplete ones. **Provenance:** symGCAD binary `63c949728334` (303 groups) run 2026-10-12T14:28–14:36 UTC; native launch 2026-10-12T10:27 UTC, upstream `fb6ebe3`. Every row's binary hash, input hash, date and record path are in `report-data.json`.

1.1 All 331 problems

#	Problem	Scope	Status	Method · ordering	Order (H = anchor)	Cells	Solve s	Verify s	Cache	Native s	Ratio
1	finiteC_5_31_6_0__2_1_1_1_1_0_0_0_0/fixed	fixed	verified	monotone · mods	x4 x3 x2 x1 x0 · H=x4	1	0.0400	0.0276	cold	1.33	0.030
2	finiteB_4_197_5_0__2_0_1_0_0_0_1_1_0/fixed	fixed	verified	monotone · mods	x0 x7 x6 x2 · H=x0	13	0.0404	0.0276	cold	1.14	0.035
3	finiteAx12_5_171_6_0__2_1_0_1_0_1_0_1_0/fixed	fixed	verified	monotone · mods	x0 x3 x1 x7 x5 · H=x0	1	0.0422	0.0277	cold	1.35	0.031
4	finiteE_2_5__3_0__2_0_1_0/fixed	fixed	verified	monotone · mods	x0 x2 · H=x0	3	0.0424	0.0275	cold	1.40	0.030
5	finiteA_4_278_5_0__0_2_1_0_1_0_0_0_1/fixed	fixed	verified	monotone · mods	x4 x2 x8 x1 · H=x4	5	0.0430	0.0273	cold	1.09	0.039
6	finiteB_5_279_6_0__2_1_1_0_1_0_0_0_1/fixed	fixed	verified	monotone · mods	x4 x2 x0 x1 x8 · H=x4	5†	0.0431	0.0270	cold	1.32	0.033
7	finiteE_2_9__3_0__2_0_0_1/fixed	fixed	verified	monotone · mods	x0 x3 · H=x0	1	0.0434	0.0296	cold	1.11	0.039
8	finiteAx12_5_31_6_0__2_1_1_1_1_0_0_0_0/fixed	fixed	verified	monotone · mods	x4 x2 x0 x3 x1 · H=x4	7	0.0436	0.0273	cold	1.64	0.027
9	finiteA_4_23_5_0__2_1_1_0_1_0_0_0_0/fixed	fixed	verified	monotone · mods	x4 x2 x0 x1 · H=x4	6	0.0439	0.0273	cold	1.40	0.031
10	finiteAx12_3_296_4_0__0_0_0_2_0_1_0_0_1/fixed	fixed	verified	monotone · mods	x3 x8 x5 · H=x3	1	0.0439	0.0273	cold	1.40	0.031
11	finiteA_5_121_6_0__2_0_0_1_1_1_1_0_0/fixed	fixed	verified	monotone · mods	x4 x6 x3 x5 x0 · H=x4	6†	0.0441	0.0274	cold	1.68	0.026
12	finiteA_5_310_6_0__0_2_1_0_1_1_0_0_1/symbolic	symbolic	verified	monotone · mods	s x4 x2 x8 x5 x1 · H=x4	6	0.0442	0.0288	cold	1.30	0.034
13	finiteA_5_155_6_0__2_1_0_1_1_0_0_1_0/fixed	fixed	verified	monotone · mods	x4 x3 x1 x0 x7 · H=x4	1	0.0446	0.0276	cold	1.66	0.027
14	finiteA_6_318_7_0__0_2_1_1_1_1_0_0_1/fixed	fixed	verified	monotone · mods	x4 x2 x8 x3 x1 x5 · H=x4	7	0.0446	0.0272	cold	2.31	0.019
15	finiteE_3_11__3_0__1_1_0_1/symbolic	symbolic	verified	monotone · mods	s t x0 x3 x1 · H=x0	8	0.0446	0.0285	cold	1.37	0.033
16	finiteAx12_6_318_7_0__0_2_1_1_1_1_0_0_1/fixed	fixed	verified	monotone · mods	x4 x2 x8 x3 x1 x5 · H=x4	7	0.0447	0.0270	cold	2.38	0.019

#	Problem	Scope	Status	Method · ordering	Order (H = anchor)	Cells	Solve s	Verify s	Cache	Native s	Ratio
17	finiteAx12_5_313_6_0__2_0_0_1_1_1_0_0_1/fixed	fixed	verified	monotone · mods	x4 x5 x8 x3 x0 · H=x4	1	0.0448	0.0292	cold	1.56	0.029
18	finiteA_4_29_5_0__2_0_1_1_1_0_0_0_0/fixed	fixed	verified	monotone · mods	x4 x2 x0 x3 · H=x4	6	0.0450	0.0277	cold	1.30	0.035
19	finiteA_2_17_3_0__2_0_0_0_1_0_0_0_0/symbolic	symbolic	verified	monotone · mods	s x4 x0 · H=x4	1	0.0452	0.0286	cold	1.11	0.041
20	finiteA_4_297_5_0__2_0_0_1_0_1_0_0_1/fixed	fixed	verified	monotone · mods	x0 x3 x8 x5 · H=x0	1	0.0455	0.0277	cold	1.15	0.040
21	finiteAx12_5_155_6_0__2_1_0_1_1_0_0_1_0/fixed	fixed	verified	monotone · mods	x4 x3 x1 x0 x7 · H=x4	1	0.0455	0.0271	cold	1.11	0.041
22	test_unbounded/solve/1	core	verified	optimized · mods	x · H=x	1	0.0461	0.0288	cold	0.540	0.085
23	finiteC_6_95_7_0__2_1_1_1_1_0_1_0_0/fixed	fixed	verified	monotone · mods	x4 x3 x2 x1 x0 x6 · H=x4	1	0.0462	0.0282	cold	1.51	0.031
24	finiteA_5_376_6_0__0_0_0_2_1_1_1_0_1/fixed	fixed	verified	monotone · mods	x4 x6 x8 x5 x3 · H=x4	5†	0.0462	0.0275	cold	1.08	0.043
25	finiteB_3_21_4_0__2_0_1_0_1_0_0_0_0/fixed	fixed	verified	monotone · mods	x4 x2 x0 · H=x4	3	0.0463	0.0276	cold	1.42	0.033
26	finiteB_5_339_6_0__2_1_0_0_1_0_1_0_1/fixed	fixed	verified	monotone · mods	x4 x8 x6 x0 x1 · H=x4	6	0.0464	0.0289	cold	1.34	0.035
27	finiteA_4_170_5_0__0_2_0_1_0_1_0_1_0/fixed	fixed	verified	monotone · mods	x1 x7 x5 x3 · H=x1	1	0.0464	0.0275	cold	1.35	0.034
28	finiteAx12_5_316_10_0__0_0_6_1_1_1_0_0_1/fixed	fixed	verified	monotone · mods	x4 x2 x8 x5 x3 · H=x4	6	0.0464	0.0271	cold	1.29	0.036
29	finiteCx12_5_158_6_0__0_2_1_1_1_0_0_1_0/fixed	fixed	verified	monotone · mods	x4 x3 x2 x1 x7 · H=x4	1	0.0465	0.0273	cold	1.09	0.043
30	finiteB_5_213_6_0__2_0_1_0_1_0_1_1_0/fixed	fixed	verified	monotone · mods	x4 x7 x6 x2 x0 · H=x4	10	0.0466	0.0267	cold	1.30	0.036
31	finiteE_2_5__3_0__2_0_1_0/symbolic	symbolic	verified	monotone · mods	s x0 x2 · H=x0	3	0.0466	0.0292	cold	1.09	0.043
32	finiteAx12_6_159_7_0__2_1_1_1_1_0_0_1_0/fixed	fixed	verified	monotone · mods	x4 x2 x0 x3 x1 x7 · H=x4	8†	0.0467	0.0270	cold	1.54	0.030
33	finiteA_5_93_6_0__2_0_1_1_1_0_1_0_0/fixed	fixed	verified	monotone · mods	x4 x6 x2 x0 x3 · H=x4	19†	0.0467	0.0268	cold	1.07	0.044
34	test_no_real_roots/solve/1	core	verified	optimized · mods	x	1	0.0468	0.0286	cold	0.540	0.087
35	finiteA_5_309_6_0__2_0_1_0_1_1_0_0_1/fixed	fixed	verified	monotone · mods	x4 x2 x0 x8 x5 · H=x4	25†	0.0468	0.0269	cold	1.30	0.036
36	finiteA_5_155_6_0__2_1_0_1_1_0_0_1_0/symbolic	symbolic	verified	monotone · mods	s t x4 x3 x1 x0 x7 · H=x4	10†	0.0469	0.0270	cold	1.62	0.029
37	test_input_unmodified/order/3	core	verified	optimized · mods	t u z y x	2	0.0470	0.0291	cold	0.600	0.078
38	finiteE_2_10__3_0__0_2_0_1/symbolic	symbolic	verified	monotone · mods	s t x1 x3 · H=x1	5	0.0471	0.0284	cold	1.10	0.043
39	finiteC_6_438_7_0__0_2_1_0_1_1_0_1_1/fixed	fixed	verified	monotone · mods	x4 x8 x7 x5 x2 x1 · H=x4	1	0.0471	0.0274	cold	1.57	0.030
40	finiteAx12_5_314_6_0__0_2_0_1_1_1_0_0_1/fixed	fixed	verified	monotone · mods	x4 x5 x8 x3 x1 · H=x4	1	0.0471	0.0274	cold	1.12	0.042
41	finiteAx12_4_312_5_0__0_0_0_2_1_1_0_0_1/fixed	fixed	verified	monotone · mods	x4 x5 x8 x3 · H=x4	1	0.0471	0.0271	cold	1.41	0.033
42	finiteA_5_310_6_0__0_2_1_0_1_1_0_0_1/fixed	fixed	verified	monotone · mods	x4 x2 x8 x5 x1 · H=x4	6	0.0472	0.0276	cold	1.37	0.034
43	finiteC_5_412_6_0__0_0_2_1_1_0_0_1_1/fixed	fixed	verified	monotone · mods	x4 x8 x7 x3 x2 · H=x4	1	0.0473	0.0298	cold	1.19	0.040
44	finiteAx12_5_301_6_0__2_0_1_1_0_1_0_0_1/fixed	fixed	verified	monotone · mods	x0 x2 x8 x3 x5 · H=x0	5†	0.0473	0.0268	cold	1.08	0.044
45	finiteA_4_89_5_0__2_0_0_1_1_0_1_0_0/fixed	fixed	verified	monotone · mods	x4 x6 x3 x0 · H=x4	5†	0.0473	0.0275	cold	1.48	0.032
46	finiteA_5_345_6_0__2_0_0_1_1_0_1_0_1/fixed	fixed	verified	monotone · mods	x4 x6 x0 x8 x3 · H=x4	12†	0.0474	0.0271	cold	1.60	0.030
47	finiteC_5_227_6_0__2_1_0_0_0_1_1_1_0/fixed	fixed	verified	monotone · mods	x0 x7 x6 x5 x1 · H=x0	1	0.0474	0.0274	cold	1.32	0.036
48	finiteAx12_6_190_7_0__0_2_1_1_1_1_0_1_0/fixed	fixed	verified	monotone · mods	x4 x7 x5 x3 x2 x1 · H=x4	1	0.0475	0.0273	cold	1.57	0.030
49	test_gluza_ex33_trivial/order/1	core	verified	optimized · mods	x4 x7 x6 x5 x3 x2 x1 x0 · H=x4	0	0.0475	0.0290	cold	timeout	—
50	finiteCx12_6_438_7_0__0_2_1_0_1_1_0_1_1/fixed	fixed	verified	monotone · mods	x4 x8 x7 x5 x2 x1 · H=x4	1	0.0476	0.0279	cold	1.51	0.032
51	finiteB_4_85_5_0__2_0_1_0_1_0_1_0_0/fixed	fixed	verified	monotone · mods	x4 x2 x0 x6 · H=x4	4†	0.0477	0.0275	cold	1.29	0.037
52	finiteA_4_278_5_0__0_2_1_0_1_0_0_0_1/symbolic	symbolic	verified	monotone · mods	s x4 x2 x8 x1 · H=x4	5	0.0477	0.0276	cold	1.14	0.042
53	finiteCx12_6_231_7_0__2_1_1_0_0_1_1_1_0/fixed	fixed	verified	monotone · mods	x0 x2 x1 x7 x6 x5 · H=x0	1	0.0477	0.0280	cold	2.90	0.016
54	test_ex44_T_C1/solve/1	core	verified	optimized · mods	z y x	3	0.0477	0.0292	cold	0.590	0.081
55	finiteAx12_4_297_5_0__2_0_0_1_0_1_0_0_1/fixed	fixed	verified	monotone · mods	x0 x3 x8 x5 · H=x0	1	0.0477	0.0276	cold	1.43	0.033
56	test_input_unmodified/solve/4	core	verified	optimized · mods	x z y u t	2	0.0479	0.0291	cold	0.610	0.078
57	test_hypercube_3/solve/1	core	verified	optimized · mods	z y x	1	0.0479	0.0287	cold	0.540	0.089
58	finiteE_3_13__3_0__1_0_1_1/fixed	fixed	verified	monotone · mods	x0 x2 x3 · H=x0	4	0.0479	0.0278	cold	1.08	0.044
59	finiteA_5_301_6_0__2_0_1_1_0_1_0_0_1/fixed	fixed	verified	monotone · mods	x0 x2 x8 x3 x5 · H=x0	5†	0.0480	0.0276	cold	1.33	0.036
60	finiteBx12_6_343_7_0__2_1_1_0_1_0_1_0_1/fixed	fixed	verified	monotone · mods	x4 x8 x6 x2 x0 x1 · H=x4	15	0.0481	0.0293	cold	1.51	0.032
61	finiteA_3_296_4_0__0_0_0_2_0_1_0_0_1/symbolic	symbolic	verified	monotone · mods	s t x3 x5 x8 · H=x3	6	0.0481	0.0271	cold	1.08	0.045
62	finiteC_6_231_7_0__2_1_1_0_0_1_1_1_0/fixed	fixed	verified	monotone · mods	x0 x2 x1 x7 x6 x5 · H=x0	1	0.0482	0.0276	cold	3.22	0.015
63	test_x2_minus_y/solve/1	core	verified	optimized · mods	x y	1	0.0482	0.0277	cold	0.540	0.089
64	finiteBx12_5_339_6_0__2_1_0_0_1_0_1_0_1/fixed	fixed	verified	monotone · mods	x4 x8 x6 x0 x1 · H=x4	6	0.0482	0.0287	cold	1.33	0.036
65	test_triangle2l_split/solve/3	core	verified	optimized · mods	x4 x3 x5 x2 x1 x0 · H=x4	6	0.0482	0.0293	cold	0.540	0.089
66	finiteA_5_313_6_0__2_0_0_1_1_1_0_0_1/symbolic	symbolic	verified	monotone · mods	s t x4 x5 x3 x8 x0 · H=x4	10	0.0482	0.0296	cold	1.51	0.032
67	finiteA_3_280_4_0__0_0_0_2_1_0_0_0_1/symbolic	symbolic	verified	monotone · mods	s x4 x8 x3 · H=x4	1	0.0483	0.0274	cold	1.07	0.045
68	finiteA_4_154_5_0__0_2_0_1_1_0_0_1_0/symbolic	symbolic	verified	monotone · mods	s t x4 x3 x1 x7 · H=x4	9†	0.0483	0.0291	cold	1.10	0.044
69	finiteA_4_293_5_0__2_0_1_0_0_1_0_0_1/fixed	fixed	verified	monotone · mods	x0 x2 x8 x5 · H=x0	4†	0.0483	0.0276	cold	1.31	0.037
70	finiteC_6_470_10_0__0_5_1_0_1_0_1_1_1/fixed	fixed	verified	monotone · mods	x4 x6 x2 x8 x7 x1 · H=x4	7	0.0483	0.0270	cold	1.54	0.031
71	test_hypercube_4/solve/1	core	verified	optimized · mods	z y x	1	0.0484	0.0288	cold	0.540	0.090
72	finiteCx12_6_215_11_0__6_1_1_0_1_0_1_1_0/fixed	fixed	verified	monotone · mods	x4 x2 x1 x0 x7 x6 · H=x4	1	0.0484	0.0274	cold	5.32	0.0091
73	finiteA_5_121_6_0__2_0_0_1_1_1_1_0_0/symbolic	symbolic	verified	monotone · mods	s x4 x6 x3 x5 x0 · H=x4	6†	0.0485	0.0273	cold	1.08	0.045

#	Problem	Scope	Status	Method · ordering	Order (H = anchor)	Cells	Solve s	Verify s	Cache	Native s	Ratio
74	finiteA_4_344_5_0__0_0_0_2_1_0_1_0_1/symbolic	symbolic	verified	monotone · mods	s x4 x6 x8 x3 · H=x4	4†	0.0486	0.0288	cold	1.10	0.044
75	finiteC_5_214_10_0__0_6_1_0_1_0_1_1_0/fixed	fixed	verified	monotone · mods	x4 x6 x7 x2 x1 · H=x4	1	0.0486	0.0283	cold	1.50	0.032
76	finiteA_3_276_4_0__0_0_2_0_1_0_0_0_1/symbolic	symbolic	verified	monotone · mods	s x4 x2 x8 · H=x4	4	0.0486	0.0272	cold	1.08	0.045
77	test_x_y_z_minus_1/solve/1	core	verified	optimized · mods	x y z	2	0.0487	0.0289	cold	0.550	0.088
78	test_hypercube_2/solve/1	core	verified	optimized · mods	y x z	1	0.0487	0.0282	cold	0.520	0.094
79	finiteA_6_190_7_0__0_2_1_1_1_1_0_1_0/fixed	fixed	verified	monotone · mods	x4 x7 x5 x3 x2 x1 · H=x4	1	0.0487	0.0274	cold	1.57	0.031
80	finiteF_4_15__4_0__1_1_1_1/fixed	fixed	verified	monotone · mods	x0 x3 x2 x1 · H=x0	1	0.0487	0.0272	cold	1.13	0.043
81	test_greedy_sotd_order/order/4	core	verified	optimized · mods	x y	7	0.0487	0.0291	cold	0.540	0.090
82	finiteA_5_157_6_0__2_0_1_1_1_0_0_1_0/symbolic	symbolic	verified	monotone · mods	s x4 x2 x0 x3 x7 · H=x4	7†	0.0488	0.0272	cold	1.52	0.032
83	finiteAx12_6_414_10_0__0_5_1_1_1_0_0_1_1/fixed	fixed	verified	monotone · mods	x4 x2 x8 x3 x1 x7 · H=x4	7	0.0488	0.0285	cold	4.81	0.010
84	finiteA_4_154_5_0__0_2_0_1_1_0_0_1_0/fixed	fixed	verified	monotone · mods	x4 x7 x3 x1 · H=x4	1	0.0488	0.0275	cold	1.34	0.036
85	test_massive_triangle/solve/1	core	verified	optimized · mods	x1 x0 s	6	0.0489	0.0289	cold	0.570	0.086
86	finiteA_5_31_6_0__2_1_1_1_1_0_0_0_0/fixed	fixed	verified	monotone · mods	x4 x2 x0 x3 x1 · H=x4	7	0.0489	0.0273	cold	1.12	0.044
87	finiteB_6_343_7_0__2_1_1_0_1_0_1_0_1/fixed	fixed	verified	monotone · mods	x4 x8 x6 x2 x0 x1 · H=x4	15	0.0490	0.0290	cold	1.30	0.038
88	finiteA_5_309_6_0__2_0_1_0_1_1_0_0_1/symbolic	symbolic	verified	monotone · mods	s x4 x2 x0 x8 x5 · H=x4	46†	0.0490	0.0281	cold	1.73	0.028
89	finiteE_2_9__3_0__2_0_0_1/symbolic	symbolic	verified	monotone · mods	s x0 x3 · H=x0	1	0.0490	0.0296	cold	1.13	0.043
90	finiteAx12_5_410_10_0__0_6_0_1_1_0_0_1_1/fixed	fixed	verified	monotone · mods	x4 x7 x8 x3 x1 · H=x4	1	0.0491	0.0273	cold	1.07	0.046
91	test_gluza_ex33/order/1	core	verified	linear_negative · given	x6 x1 x4 x0 x2 x7 x5 x3	3	0.0492	0.0301	cold	timeout	—
92	test_ex412_box/solve/1	core	verified	optimized · mods	x4 x1 p1s s13 x3 x2 s12	2	0.0492	0.0286	cold	0.560	0.088
93	test_cube/solve/1	core	verified	optimized · mods	z y x	1	0.0492	0.0295	cold	0.570	0.086
94	finiteE_3_13__3_0__1_0_1_1/symbolic	symbolic	verified	monotone · mods	s x0 x2 x3 · H=x0	4	0.0494	0.0288	cold	1.12	0.044
95	finiteA_4_27_5_0__2_1_0_1_1_0_0_0_0/symbolic	symbolic	verified	monotone · mods	s t x4 x3 x1 x0 · H=x4	9	0.0494	0.0295	cold	1.09	0.045
96	finiteE_3_7__3_0__1_1_1_0/fixed	fixed	verified	monotone · mods	x0 x2 x1 · H=x0	4	0.0494	0.0271	cold	1.11	0.044
97	finiteA_5_87_6_0__2_1_1_0_1_0_1_0_0/fixed	fixed	verified	monotone · mods	x4 x6 x2 x0 x1 · H=x4	19†	0.0495	0.0281	cold	1.08	0.046
98	finiteA_6_414_10_0__0_5_1_1_1_0_0_1_1/fixed	fixed	verified	monotone · mods	x4 x2 x8 x3 x1 x7 · H=x4	7	0.0496	0.0277	cold	5.23	0.0095
99	test_ex44_B1_B4/solve/1	core	verified	optimized · mods	z y x	0	0.0496	0.0287	cold	0.550	0.090
100	test_pentabox_exb/order/1	core	verified	optimized · mods	x4 x7 x6 x5 x3 x2 x1 x0 · H=x4	7	0.0496	0.0294	cold	0.740	0.067
101	finiteA_3_25_4_0__2_0_0_1_1_0_0_0_0/symbolic	symbolic	verified	monotone · mods	s x4 x3 x0 · H=x4	1	0.0496	0.0282	cold	1.11	0.045
102	test_greedy_mods_order/order/1	core	verified	optimized · mods	x y z	6	0.0497	0.0289	cold	0.530	0.094
103	finiteA_3_21_4_0__2_0_1_0_1_0_0_0_0/symbolic	symbolic	verified	monotone · mods	s x4 x2 x0 · H=x4	5	0.0497	0.0279	cold	1.20	0.041
104	test_gluza_ex33_trivial/solve/3	core	verified	optimized · mods	x4 x7 x6 x5 x3 x2 x1 x0 · H=x4	1	0.0497	0.0295	cold	timeout	—
105	test_ex44_B1_C2/solve/1	core	verified	optimized · mods	z y x	0	0.0497	0.0292	cold	0.540	0.092
106	finiteA_4_85_5_0__2_0_1_0_1_0_1_0_0/fixed	fixed	verified	monotone · mods	x4 x6 x2 x0 · H=x4	15	0.0497	0.0271	cold	1.34	0.037
107	finiteC_5_158_6_0__0_2_1_1_1_0_0_1_0/fixed	fixed	verified	monotone · mods	x4 x3 x2 x1 x7 · H=x4	1	0.0497	0.0286	cold	1.31	0.038
108	test_ex411_box/solve/2	core	verified	optimized · mods	x4 x3 x2 x1 s13 s12 · H=x4	1	0.0498	0.0276	cold	0.580	0.086
109	finiteAx12_4_282_5_0__0_2_0_1_1_0_0_0_1/fixed	fixed	verified	monotone · mods	x4 x8 x3 x1 · H=x4	1	0.0498	0.0270	cold	1.13	0.044
110	finiteE_3_7__3_0__1_1_1_0/symbolic	symbolic	verified	monotone · mods	s x0 x2 x1 · H=x0	4	0.0499	0.0285	cold	1.08	0.046
111	finiteA_4_153_5_0__2_0_0_1_1_0_0_1_0/symbolic	symbolic	verified	monotone · mods	s x4 x7 x3 x0 · H=x4	1	0.0500	0.0285	cold	1.10	0.045
112	finiteA_4_153_5_0__2_0_0_1_1_0_0_1_0/fixed	fixed	verified	monotone · mods	x4 x7 x3 x0 · H=x4	1	0.0501	0.0274	cold	1.43	0.035
113	finiteAx12_5_376_6_0__0_0_0_2_1_1_1_0_1/fixed	fixed	verified	monotone · mods	x4 x6 x8 x5 x3 · H=x4	5†	0.0502	0.0286	cold	1.29	0.039
114	finiteE_3_11__3_0__1_1_0_1/fixed	fixed	verified	monotone · mods	x0 x3 x1 · H=x0	1	0.0502	0.0280	cold	1.61	0.031
115	finiteB_4_277_5_0__2_0_1_0_1_0_0_0_1/fixed	fixed	verified	monotone · mods	x4 x2 x0 x8 · H=x4	4†	0.0502	0.0289	cold	1.11	0.045
116	test_pentabox_exb/solve/3	core	verified	optimized · mods	x4 x7 x6 x5 x3 x2 x1 x0 · H=x4	3	0.0503	0.0293	cold	0.980	0.051
117	finiteB_5_341_6_0__2_0_1_0_1_0_1_0_1/fixed	fixed	verified	monotone · mods	x4 x8 x6 x2 x0 · H=x4	11	0.0503	0.0288	cold	1.33	0.038
118	finiteB_5_453_6_0__2_0_1_0_0_0_1_1_1/fixed	fixed	verified	monotone · mods	x0 x8 x7 x6 x2 · H=x0	77	0.0503	0.0290	cold	1.66	0.030
119	finiteBx12_5_279_6_0__2_1_1_0_1_0_0_0_1/symbolic	symbolic	verified	monotone · mods	s t x4 x8 x2 x1 x0 · H=x4	18	0.0503	0.0296	cold	1.10	0.046
120	finiteA_4_23_5_0__2_1_1_0_1_0_0_0_0/symbolic	symbolic	verified	monotone · mods	s x4 x2 x0 x1 · H=x4	6	0.0504	0.0276	cold	1.09	0.046
121	finiteC_6_476_10_0__0_0_5_1_1_0_1_1_1/fixed	fixed	verified	monotone · mods	x4 x6 x2 x8 x7 x3 · H=x4	7	0.0504	0.0273	cold	1.56	0.032
122	finiteA_4_282_5_0__0_2_0_1_1_0_0_0_1/symbolic	symbolic	verified	monotone · mods	s t x4 x3 x1 x8 · H=x4	36	0.0504	0.0293	cold	1.08	0.047
123	finiteAx12_5_286_6_0__0_2_1_1_1_0_0_0_1/fixed	fixed	verified	monotone · mods	x4 x2 x8 x3 x1 · H=x4	6	0.0505	0.0283	cold	1.57	0.032
124	finiteBx12_5_279_6_0__2_1_1_0_1_0_0_0_1/fixed	fixed	verified	monotone · mods	x4 x2 x0 x1 x8 · H=x4	5†	0.0505	0.0293	cold	1.13	0.045
125	finiteB_5_213_6_0__2_0_1_0_1_0_1_1_0/symbolic	symbolic	verified	monotone · mods	s x4 x7 x6 x2 x0 · H=x4	10	0.0505	0.0293	cold	1.47	0.034
126	test_ex422_bubble/solve/1	core	verified	optimized · mods	x1 x2 b · H=x1	2	0.0505	0.0276	cold	0.550	0.092
127	finiteA_4_344_5_0__0_0_0_2_1_0_1_0_1/fixed	fixed	verified	monotone · mods	x4 x6 x8 x3 · H=x4	4†	0.0505	0.0276	cold	1.08	0.047
128	finiteA_3_26_4_0__0_2_0_1_1_0_0_0_0/symbolic	symbolic	verified	monotone · mods	s t x4 x3 x1 · H=x4	8	0.0506	0.0270	cold	1.09	0.046
129	test_cylinder_3d/solve/1	core	verified	optimized · mods	z y x	1	0.0507	0.0276	cold	0.530	0.096
130	finiteAx12_4_154_5_0__0_2_0_1_1_0_0_1_0/symbolic	symbolic	verified	monotone · mods	t s x4 x3 x1 x7 · H=x4	9†	0.0508	0.0294	cold	1.38	0.037

#	Problem	Scope	Status	Method · ordering	Order (H = anchor)	Cells	Solve s	Verify s	Cache	Native s	Ratio
131	finiteA_4_284_5_0__0_0_2_1_1_0_0_0_1/symbolic	symbolic	verified	monotone · mods	s x4 x2 x8 x3 · H=x4	5	0.0509	0.0287	cold	1.13	0.045
132	finiteE_4_15__4_0__1_1_1_1/fixed	fixed	verified	monotone · mods	x0 x2 x3 x1 · H=x0	5	0.0510	0.0272	cold	1.10	0.046
133	test_y2_minus_x/solve/1	core	verified	optimized · mods	y x	1	0.0511	0.0287	cold	0.550	0.093
134	finiteA_4_284_5_0__0_0_2_1_1_0_0_0_1/fixed	fixed	verified	monotone · mods	x4 x2 x8 x3 · H=x4	5	0.0511	0.0275	cold	1.44	0.035
135	test_massive_triangle/solve/2	core	verified	optimized · mods	x1 x0 s	4	0.0511	0.0273	cold	0.600	0.085
136	test_boundary/solve/1	core	verified	optimized · mods	x	0	0.0511	0.0289	cold	0.550	0.093
137	finiteA_4_312_5_0__0_0_0_2_1_1_0_0_1/symbolic	symbolic	verified	monotone · mods	s t x4 x5 x3 x8 · H=x4	9	0.0511	0.0269	cold	1.41	0.036
138	finiteE_2_10__3_0__0_2_0_1/fixed	fixed	verified	monotone · mods	x1 x3 · H=x1	1	0.0511	0.0274	cold	1.10	0.046
139	finiteCx12_6_476_10_0__0_0_5_1_1_0_1_1_1/fixed	fixed	verified	monotone · mods	x4 x6 x2 x8 x7 x3 · H=x4	7	0.0513	0.0282	cold	1.52	0.034
140	test_input_unmodified/order/2	core	verified	optimized · mods	t u z y x	2	0.0513	0.0286	cold	0.560	0.092
141	finiteA_4_312_5_0__0_0_0_2_1_1_0_0_1/fixed	fixed	verified	monotone · mods	x4 x5 x8 x3 · H=x4	1	0.0513	0.0277	cold	1.08	0.048
142	finiteAx12_5_155_6_0__2_1_0_1_1_0_0_1_0/symbolic	symbolic	verified	monotone · mods	t s x4 x3 x1 x0 x7 · H=x4	10†	0.0513	0.0297	cold	1.30	0.039
143	finiteAx12_4_282_5_0__0_2_0_1_1_0_0_0_1/symbolic	symbolic	verified	monotone · mods	t s x4 x3 x1 x8 · H=x4	36	0.0514	0.0293	cold	1.10	0.047
144	finiteAx12_4_27_5_0__2_1_0_1_1_0_0_0_0/fixed	fixed	verified	monotone · mods	x4 x3 x1 x0 · H=x4	1	0.0514	0.0288	cold	1.09	0.047
145	finiteC_6_215_11_0__6_1_1_0_1_0_1_1_0/fixed	fixed	verified	monotone · mods	x4 x2 x1 x0 x7 x6 · H=x4	1	0.0514	0.0281	cold	5.55	0.0093
146	finiteA_4_282_5_0__0_2_0_1_1_0_0_0_1/fixed	fixed	verified	monotone · mods	x4 x8 x3 x1 · H=x4	1	0.0515	0.0277	cold	1.14	0.045
147	finiteA_4_27_5_0__2_1_0_1_1_0_0_0_0/fixed	fixed	verified	monotone · mods	x4 x3 x1 x0 · H=x4	1	0.0515	0.0277	cold	1.15	0.045
148	test_ex44_B1_C1/solve/1	core	verified	optimized · mods	z y x	2	0.0515	0.0293	cold	0.540	0.095
149	finiteA_5_286_6_0__0_2_1_1_1_0_0_0_1/fixed	fixed	verified	monotone · mods	x4 x2 x8 x3 x1 · H=x4	6	0.0516	0.0276	cold	1.38	0.037
150	finiteB_5_279_6_0__2_1_1_0_1_0_0_0_1/symbolic	symbolic	verified	monotone · mods	t s x4 x8 x2 x1 x0 · H=x4	18	0.0517	0.0296	cold	1.59	0.032
151	finiteAx12_4_312_5_0__0_0_0_2_1_1_0_0_1/symbolic	symbolic	verified	monotone · mods	t s x4 x5 x3 x8 · H=x4	9	0.0517	0.0291	cold	1.10	0.047
152	finiteA_5_316_10_0__0_0_6_1_1_1_0_0_1/fixed	fixed	verified	monotone · mods	x4 x2 x8 x5 x3 · H=x4	6	0.0518	0.0274	cold	1.13	0.046
153	test_hypercube_5/solve/1	core	verified	optimized · mods	z y x	2	0.0519	0.0276	cold	0.590	0.088
154	finiteB_4_85_5_0__2_0_1_0_1_0_1_0_0/symbolic	symbolic	verified	monotone · mods	s x4 x2 x0 x6 · H=x4	4†	0.0520	0.0295	cold	1.07	0.049
155	finiteAx12_4_297_5_0__2_0_0_1_0_1_0_0_1/symbolic	symbolic	verified	monotone · mods	t s x0 x5 x3 x8 · H=x0	9	0.0520	0.0289	cold	1.07	0.049
156	test_empty_3d/solve/1	core	verified	optimized · mods	z y x	0	0.0520	0.0295	cold	0.550	0.095
157	test_ex422_bubble/solve/2	core	verified	optimized · mods	x1 x2 b · H=x1	2	0.0521	0.0274	cold	0.570	0.091
158	finiteAx12_6_377_10_0__5_0_0_1_1_1_1_0_1/fixed	fixed	verified	monotone · mods	x4 x6 x0 x8 x5 x3 · H=x4	15†	0.0521	0.0286	cold	2.28	0.023
159	test_ex44_B1_B2/solve/1	core	verified	optimized · mods	z y x	1	0.0522	0.0292	cold	0.530	0.098
160	finiteA_4_85_5_0__2_0_1_0_1_0_1_0_0/symbolic	symbolic	verified	monotone · mods	s x4 x6 x2 x0 · H=x4	15	0.0522	0.0289	cold	1.53	0.034
161	test_gluza_ex30/order/1	core	verified	linear_negative · given	x2 x6 x7 x5 x1 x4 x0	1	0.0523	0.0298	cold	timeout	—
162	finiteB_4_197_5_0__2_0_1_0_0_0_1_1_0/symbolic	symbolic	verified	monotone · mods	s x0 x7 x6 x2 · H=x0	13	0.0524	0.0289	cold	1.34	0.039
163	test_input_unmodified/order/1	core	verified	optimized · mods	t u z y x	2	0.0525	0.0290	cold	0.540	0.097
164	finiteA_5_314_6_0__0_2_0_1_1_1_0_0_1/fixed	fixed	verified	monotone · mods	x4 x5 x8 x3 x1 · H=x4	1	0.0527	0.0276	cold	1.60	0.033
165	finiteB_3_21_4_0__2_0_1_0_1_0_0_0_0/symbolic	symbolic	verified	monotone · mods	s x4 x2 x0 · H=x4	3	0.0527	0.0291	cold	1.09	0.048
166	finiteAx12_4_27_5_0__2_1_0_1_1_0_0_0_0/symbolic	symbolic	verified	monotone · mods	t s x4 x3 x1 x0 · H=x4	9	0.0528	0.0284	cold	1.09	0.048
167	finiteA_5_171_6_0__2_1_0_1_0_1_0_1_0/fixed	fixed	verified	monotone · mods	x0 x3 x1 x7 x5 · H=x0	1	0.0529	0.0292	cold	1.09	0.049
168	finiteA_5_412_6_0__0_0_2_1_1_0_0_1_1/fixed	fixed	verified	monotone · mods	x4 x2 x8 x7 x3 · H=x4	6	0.0531	0.0270	cold	1.11	0.048
169	finiteA_3_296_4_0__0_0_0_2_0_1_0_0_1/fixed	fixed	verified	monotone · mods	x3 x8 x5 · H=x3	1	0.0532	0.0277	cold	1.13	0.047
170	test_ex411_box/solve/1	core	verified	optimized · mods	x4 x3 x2 x1 s13 s12	1	0.0533	0.0275	cold	0.600	0.089
171	finiteA_5_313_6_0__2_0_0_1_1_1_0_0_1/fixed	fixed	verified	monotone · mods	x4 x5 x8 x3 x0 · H=x4	1	0.0533	0.0276	cold	1.11	0.048
172	test_quadrant/solve/1	core	verified	optimized · mods	x y · H=x	1	0.0534	0.0285	cold	0.560	0.095
173	finiteA_5_157_6_0__2_0_1_1_1_0_0_1_0/fixed	fixed	verified	monotone · mods	x4 x2 x0 x3 x7 · H=x4	7†	0.0535	0.0275	cold	1.08	0.050
174	test_greedy_sotd_order/solve/2	core	verified	optimized · mods	x y	7	0.0536	0.0291	cold	0.540	0.099
175	finiteA_5_410_10_0__0_6_0_1_1_0_0_1_1/fixed	fixed	verified	monotone · mods	x4 x7 x8 x3 x1 · H=x4	1	0.0538	0.0273	cold	1.31	0.041
176	finiteA_4_89_5_0__2_0_0_1_1_0_1_0_0/symbolic	symbolic	verified	monotone · mods	s x4 x6 x3 x0 · H=x4	5†	0.0538	0.0273	cold	1.40	0.038
177	test_pentabox_exd/solve/3	core	verified	optimized · mods	x4 x7 x6 x5 x3 x2 x1 x0 · H=x4	4	0.0539	0.0293	cold	0.730	0.074
178	test_ex44_B1_B2_B4/solve/1	core	verified	optimized · mods	z y x	0	0.0541	0.0289	cold	0.550	0.098
179	finiteA_5_412_6_0__0_0_2_1_1_0_0_1_1/symbolic	symbolic	verified	monotone · mods	s x4 x2 x8 x7 x3 · H=x4	6	0.0543	0.0298	cold	1.91	0.028
180	finiteA_4_29_5_0__2_0_1_1_1_0_0_0_0/symbolic	symbolic	verified	monotone · mods	s x4 x2 x0 x3 · H=x4	6	0.0543	0.0272	cold	1.11	0.049
181	finiteCx12_7_247_10_0__4_1_1_0_1_1_1_1_0/fixed	fixed	verified	monotone · mods	x4 x6 x5 x7 x2 x1 x0 · H=x4	1	0.0546	0.0285	cold	timeout	—
182	finiteAx12_3_296_4_0__0_0_0_2_0_1_0_0_1/symbolic	symbolic	verified	monotone · mods	t s x3 x5 x8 · H=x3	6	0.0548	0.0297	cold	1.09	0.050
183	test_pentabox_exd/order/1	core	verified	optimized · mods	x4 x7 x6 x5 x3 x2 x1 x0 · H=x4	7	0.0548	0.0287	cold	0.750	0.073
184	finiteA_4_297_5_0__2_0_0_1_0_1_0_0_1/symbolic	symbolic	verified	monotone · mods	s t x0 x5 x3 x8 · H=x0	9	0.0548	0.0269	cold	1.10	0.050
185	test_ex44_T_B2/solve/1	core	verified	optimized · mods	z y x	31	0.0550	0.0290	cold	0.550	0.100
186	finiteC_7_247_10_0__4_1_1_0_1_1_1_1_0/fixed	fixed	verified	monotone · mods	x4 x6 x5 x7 x2 x1 x0 · H=x4	1	0.0550	0.0284	cold	timeout	—
187	finiteAx12_4_170_5_0__0_2_0_1_0_1_0_1_0/fixed	fixed	verified	monotone · mods	x1 x7 x5 x3 · H=x1	1	0.0551	0.0285	cold	1.39	0.040

#	Problem	Scope	Status	Method · ordering	Order (H = anchor)	Cells	Solve s	Verify s	Cache	Native s	Ratio
188	test_hypercube_1/solve/1	core	verified	optimized · mods	z y x	1	0.0551	0.0286	cold	0.540	0.10
189	finiteE_1_1__3_0__3_0_0_0/fixed	fixed	verified	monotone · mods	x0 · H=x0	1	0.0553	0.0283	cold	1.15	0.048
190	finiteAx12_3_26_4_0__0_2_0_1_1_0_0_0_0/symbolic	symbolic	verified	monotone · mods	t s x4 x3 x1 · H=x4	8	0.0557	0.0291	cold	1.14	0.049
191	finiteB_4_277_5_0__2_0_1_0_1_0_0_0_1/symbolic	symbolic	verified	monotone · mods	s x4 x2 x0 x8 · H=x4	4†	0.0559	0.0295	cold	1.19	0.047
192	test_greedy_sotd_order/order/3	core	verified	optimized · mods	y x	9	0.0560	0.0291	cold	0.540	0.10
193	test_ex44_B1_B2_B3/solve/1	core	verified	optimized · mods	z y x	4	0.0565	0.0289	cold	0.540	0.10
194	test_unbounded_neg/solve/1	core	verified	optimized · mods	x	1	0.0571	0.0287	cold	0.540	0.11
195	finiteAx12_3_26_4_0__0_2_0_1_1_0_0_0_0/fixed	fixed	verified	monotone · mods	x4 x3 x1 · H=x4	1	0.0571	0.0274	cold	1.26	0.045
196	test_ex413_pentagon/solve/1	core	verified	optimized · mods	x3 x5 x4 x1 s51 s45 x2 s34 s23 s12	5	0.0578	0.0276	cold	0.530	0.11
197	finiteE_1_1__3_0__3_0_0_0/symbolic	symbolic	verified	monotone · mods	s x0 · H=x0	1	0.0586	0.0290	cold	1.11	0.053
198	test_ex412_box/solve/2	core	verified	optimized · mods	x4 x1 p1s s13 x3 x2 s12	1	0.0587	0.0285	cold	0.560	0.10
199	finiteAx12_4_154_5_0__0_2_0_1_1_0_0_1_0/fixed	fixed	verified	monotone · mods	x4 x7 x3 x1 · H=x4	1	0.0591	0.0273	cold	1.47	0.040
200	test_greedy_sotd_order/order/5	core	verified	optimized · mods	y x	9	0.0592	0.0288	cold	0.540	0.11
201	finiteA_6_377_10_0__5_0_0_1_1_1_1_0_1/fixed	fixed	verified	monotone · mods	x4 x6 x0 x8 x5 x3 · H=x4	15†	0.0594	0.0281	cold	2.51	0.024
202	test_ex414_bnp6/solve/2	core	verified	optimized · mods	s12 s23 x1 x6 x3 x4 x2 x5	2	0.0597	0.0288	cold	0.540	0.11
203	finiteA_4_293_5_0__2_0_1_0_0_1_0_0_1/symbolic	symbolic	verified	monotone · mods	s x0 x2 x8 x5 · H=x0	4†	0.0604	0.0275	cold	1.12	0.054
204	test_triangle2l_split/order/1	core	verified	optimized · mods	x4 x3 x5 x2 x1 x0 · H=x4	4	0.0606	0.0295	cold	0.590	0.10
205	test_greedy_sotd_order/solve/1	core	verified	optimized · mods	y x	9	0.0613	0.0291	cold	0.540	0.11
206	finiteA_6_159_7_0__2_1_1_1_1_0_0_1_0/fixed	fixed	verified	monotone · mods	x4 x2 x0 x3 x1 x7 · H=x4	8†	0.0627	0.0291	cold	1.51	0.042
207	finiteA_6_125_7_0__2_0_1_1_1_1_1_0_0/fixed	fixed	verified	monotone · mods	x4 x6 x2 x0 x5 x3 · H=x4	23†	0.0639	0.0283	cold	1.62	0.039
208	test_ex413_pentagon/solve/2	core	verified	optimized · mods	x3 x5 x4 x1 s51 s45 x2 s34 s23 s12	3	0.0640	0.0293	cold	0.540	0.12
209	finiteAx12_7_381_10_0__4_0_1_1_1_1_1_0_1/fixed	fixed	verified	monotone · given	x8 x4 x6 x0 x2 x5 x3 · H=x8	836	0.0648	0.0693	cold	timeout	—
210	test_ex415_bnp7/solve/1	core	verified	optimized · mods	s12 s23 x1 x3 x2 x4 x5 x6 x7	18	0.0651	0.0287	cold	0.560	0.12
211	finiteA_6_317_7_0__2_0_1_1_1_1_0_0_1/fixed	fixed	verified	monotone · mods	x4 x2 x0 x8 x3 x5 · H=x4	32†	0.0654	0.0286	cold	1.99	0.033
212	test_ex414_bnp6/solve/1	core	verified	optimized · mods	s12 s23 x1 x6 x3 x4 x2 x5	1	0.0660	0.0274	cold	0.580	0.11
213	finiteA_7_381_10_0__4_0_1_1_1_1_1_0_1/fixed	fixed	verified	monotone · given	x8 x4 x6 x0 x2 x5 x3 · H=x8	836	0.0673	0.0700	cold	timeout	—
214	finiteA_3_26_4_0__0_2_0_1_1_0_0_0_0/fixed	fixed	verified	monotone · mods	x4 x3 x1 · H=x4	1	0.0676	0.0276	cold	1.08	0.063
215	finiteA_3_21_4_0__2_0_1_0_1_0_0_0_0/fixed	fixed	verified	monotone · mods	x4 x2 x0 · H=x4	5	0.0678	0.0275	cold	1.16	0.058
216	finiteA_3_280_4_0__0_0_0_2_1_0_0_0_1/fixed	fixed	verified	monotone · mods	x4 x8 x3 · H=x4	1	0.0679	0.0276	cold	1.34	0.051
217	finiteA_4_170_5_0__0_2_0_1_0_1_0_1_0/symbolic	symbolic	verified	monotone · mods	s t x1 x7 x5 x3 · H=x1	151	0.0681	0.0291	cold	1.34	0.051
218	finiteD_6_373_7_0__2_0_1_0_1_1_1_0_1/fixed	fixed	verified	monotone · mods	x4 x6 x8 x5 x2 x0 · H=x4	141†	0.0681	0.0298	cold	2.53	0.027
219	finiteB_6_469_7_0__2_0_1_0_1_0_1_1_1/fixed	fixed	verified	monotone · mods	x4 x7 x6 x8 x2 x0 · H=x4	184	0.0683	0.0499	cold	1.52	0.045
220	finiteA_3_25_4_0__2_0_0_1_1_0_0_0_0/fixed	fixed	verified	monotone · mods	x4 x3 x0 · H=x4	1	0.0686	0.0276	cold	1.09	0.063
221	finiteAx12_6_317_7_0__2_0_1_1_1_1_0_0_1/fixed	fixed	verified	monotone · mods	x4 x2 x0 x8 x3 x5 · H=x4	32†	0.0687	0.0290	cold	1.73	0.040
222	finiteA_2_17_3_0__2_0_0_0_1_0_0_0_0/fixed	fixed	verified	monotone · mods	x4 x0 · H=x4	1	0.0688	0.0276	cold	1.16	0.059
223	finiteAx12_7_446_11_0__0_5_1_1_1_1_0_1_1/fixed	fixed	verified	monotone · mods	x4 x2 x8 x7 x5 x3 x1 · H=x4	8	0.0696	0.0280	cold	timeout	—
224	finiteA_3_276_4_0__0_0_2_0_1_0_0_0_1/fixed	fixed	verified	monotone · mods	x4 x2 x8 · H=x4	4	0.0696	0.0275	cold	1.07	0.065
225	failing_test_ex43/project/1	core	verified	optimized · mods	t z y x	10	0.0706	0.0498	cold	0.950	0.074
226	finiteD_6_219_7_0__2_1_0_1_1_0_1_1_0/fixed	fixed	verified	monotone · mods	x4 x3 x7 x6 x1 x0 · H=x4	262	0.0712	0.0495	cold	1.55	0.046
227	finiteAx12_4_170_5_0__0_2_0_1_0_1_0_1_0/symbolic	symbolic	verified	monotone · mods	t s x1 x7 x5 x3 · H=x1	151	0.0713	0.0293	cold	1.10	0.065
228	test_ex415_bnp7/solve/2	core	verified	optimized · mods	s12 s23 x1 x3 x2 x4 x5 x6 x7	10	0.0714	0.0296	cold	0.580	0.12
229	finiteA_5_93_6_0__2_0_1_1_1_0_1_0_0/symbolic	symbolic	verified	monotone · mods	s x4 x6 x2 x0 x3 · H=x4	19†	0.0714	0.0293	cold	1.50	0.048
230	finiteA_5_87_6_0__2_1_1_0_1_0_1_0_0/symbolic	symbolic	verified	monotone · mods	s x4 x6 x2 x0 x1 · H=x4	19†	0.0717	0.0294	cold	1.32	0.054
231	test_pentabox_exb_symb/order/1	core	verified	optimized · mods	s15 s45 s34 s23 s12 x7 x6 x5 x4 x3 x2 x1 x0	5	0.0720	0.0292	cold	timeout	—
232	finiteE_4_15__4_0__1_1_1_1/symbolic	symbolic	verified	monotone · mods	s t x0 x3 x2 x1 · H=x0	406	0.0720	0.0495	cold	1.08	0.067
233	finiteA_5_316_10_0__0_0_6_1_1_1_0_0_1/symbolic	symbolic	verified	monotone · mods	s t x4 x3 x5 x2 x8 · H=x4	151	0.0742	0.0294	cold	timeout	—
234	finiteA_5_345_6_0__2_0_0_1_1_0_1_0_1/symbolic	symbolic	verified	monotone · mods	s x4 x6 x0 x8 x3 · H=x4	12†	0.0745	0.0287	cold	1.89	0.039
235	failing_test_ex44_T_C1_B2/solve/1	core	verified	optimized · mods	z y x	57	0.0745	0.0697	cold	0.740	0.10
236	finiteA_5_314_6_0__0_2_0_1_1_1_0_0_1/symbolic	symbolic	verified	monotone · mods	s t x4 x3 x1 x5 x8 · H=x4	144	0.0746	0.0486	cold	2.31	0.032
237	finiteAx12_5_313_6_0__2_0_0_1_1_1_0_0_1/symbolic	symbolic	verified	monotone · mods	t s x4 x5 x3 x8 x0 · H=x4	10	0.0753	0.0303	cold	1.56	0.048
238	finiteA_7_446_11_0__0_5_1_1_1_1_0_1_1/fixed	fixed	verified	monotone · mods	x4 x2 x8 x7 x5 x3 x1 · H=x4	8	0.0778	0.0280	cold	timeout	—
239	finiteC_7_478_10_0__0_4_1_1_1_0_1_1_1/fixed	fixed	verified	monotone · mods	x4 x6 x2 x8 x7 x3 x1 · H=x4	8	0.0779	0.0271	cold	timeout	—
240	finiteB_5_341_6_0__2_0_1_0_1_0_1_0_1/symbolic	symbolic	verified	monotone · mods	s x4 x8 x6 x2 x0 · H=x4	11	0.0791	0.0283	cold	1.33	0.059
241	finiteAx12_5_316_10_0__0_0_6_1_1_1_0_0_1/symbolic	symbolic	verified	monotone · mods	t s x4 x3 x5 x2 x8 · H=x4	249	0.0798	0.0499	cold	timeout	—
242	test_elliptic2l_physical_symb/solve/2	core	verified	optimized · mods	x4 x0 x1 x3 x6 pp4 x5 s t x2 · H=x4	15†	0.0836	0.0303	cold	timeout	—
243	finiteCx12_7_478_10_0__0_4_1_1_1_0_1_1_1/fixed	fixed	verified	monotone · mods	x4 x6 x2 x8 x7 x3 x1 · H=x4	8	0.0873	0.0291	cold	timeout	—
244	finiteB_7_471_13_0__7_1_1_0_1_0_1_1_1/fixed	fixed	verified	monotone · mods	x4 x7 x6 x8 x2 x1 x0 · H=x4	245	0.0880	0.0491	cold	1.53	0.057

#	Problem	Scope	Status	Method · ordering	Order (H = anchor)	Cells	Solve s	Verify s	Cache	Native s	Ratio
245	test_pentabox_exd_symb/order/1	core	verified	optimized · mods	s15 s45 s23 s34 s12 x7 x6 x5 x4 x3 x2 x1 x0	152	0.0895	0.0497	cold	timeout	—
246	finiteF_4_15__4_0__1_1_1_1/symbolic	symbolic	verified	monotone · mods	t s x0 x3 x2 x1 · H=x0	348†	0.0898	0.0697	cold	1.53	0.059
247	finiteC_5_214_10_0__0_6_1_0_1_0_1_1_0/symbolic	symbolic	verified	monotone · mods	t s x4 x6 x2 x1 x7 · H=x4	354†	0.0937	0.0901	cold	timeout	—
248	finiteC_5_412_6_0__0_0_2_1_1_0_0_1_1/symbolic	symbolic	verified	monotone · mods	t s x4 x3 x2 x8 x7 · H=x4	228	0.0961	0.0696	cold	1.52	0.063
249	test_ex44_HB1_HB2_HB3/solve/1	core	verified	optimized · mods	z y x	0	0.0966	0.110	cold	0.540	0.18
250	finiteD_6_219_7_0__2_1_0_1_1_0_1_1_0/symbolic	symbolic	verified	monotone · mods	s x4 x3 x7 x6 x1 x0 · H=x4	262	0.0968	0.0496	cold	6.11	0.016
251	finiteAx12_5_314_6_0__0_2_0_1_1_1_0_0_1/symbolic	symbolic	verified	monotone · mods	t s x4 x3 x1 x5 x8 · H=x4	144	0.0969	0.0498	cold	1.94	0.050
252	finiteC_5_227_6_0__2_1_0_0_0_1_1_1_0/symbolic	symbolic	verified	monotone · mods	t s x0 x6 x5 x1 x7 · H=x0	354†	0.0994	0.0715	cold	14.5	0.0069
253	test_ex44_HT_HB2_HB3/solve/1	core	verified	optimized · mods	y x z	0	0.1000	0.0698	cold	0.750	0.13
254	finiteBx12_7_471_13_0__7_1_1_0_1_0_1_1_1/fixed	fixed	verified	monotone · mods	x4 x7 x6 x8 x2 x1 x0 · H=x4	245	0.105	0.0494	cold	1.51	0.070
255	finiteD_7_375_13_0__7_1_1_0_1_1_1_0_1/fixed	fixed	verified	monotone · mods	x4 x6 x8 x5 x2 x0 x1 · H=x4	196†	0.109	0.0283	cold	6.17	0.018
256	finiteD_7_251_12_0__6_1_0_1_1_1_1_1_0/fixed	fixed	verified	monotone · mods	x4 x3 x7 x6 x1 x0 x5 · H=x4	364	0.111	0.0493	cold	121	0.00091
257	finiteBx12_5_339_6_0__2_1_0_0_1_0_1_0_1/symbolic	symbolic	verified	monotone · mods	s t x4 x8 x6 x1 x0 · H=x4	954	0.113	0.110	cold	1.56	0.072
258	finiteB_5_453_6_0__2_0_1_0_0_0_1_1_1/symbolic	symbolic	verified	monotone · mods	s x0 x8 x7 x6 x2 · H=x0	524	0.116	0.0497	cold	1.30	0.089
259	finiteB_6_469_7_0__2_0_1_0_1_0_1_1_1/symbolic	symbolic	verified	monotone · mods	s x4 x7 x6 x8 x2 x0 · H=x4	184	0.118	0.0496	cold	2.67	0.044
260	finiteB_5_339_6_0__2_1_0_0_1_0_1_0_1/symbolic	symbolic	verified	monotone · mods	t s x4 x8 x6 x1 x0 · H=x4	1,079	0.119	0.131	cold	1.48	0.081
261	finiteAx12_5_410_10_0__0_6_0_1_1_0_0_1_1/symbolic	symbolic	verified	monotone · mods	t s x4 x7 x3 x1 x8 · H=x4	973	0.131	0.131	cold	2.27	0.058
262	finiteA_6_125_7_0__2_0_1_1_1_1_1_0_0/symbolic	symbolic	verified	monotone · mods	s x4 x6 x2 x0 x5 x3 · H=x4	23†	0.139	0.0294	cold	1.55	0.090
263	finiteCx12_5_158_6_0__0_2_1_1_1_0_0_1_0/symbolic	symbolic	verified	optimized · mods	t s x3 x2 x1 x4 x7	1,273	0.147	0.212	cold	2.31	0.064
264	finiteA_5_410_10_0__0_6_0_1_1_0_0_1_1/symbolic	symbolic	verified	monotone · mods	s t x4 x7 x3 x1 x8 · H=x4	973	0.154	0.151	cold	3.91	0.039
265	finiteBx12_6_343_7_0__2_1_1_0_1_0_1_0_1/symbolic	symbolic	verified	monotone · mods	s t x4 x8 x6 x2 x1 x0 · H=x4	1,963	0.156	0.212	cold	1.56	0.100
266	test_ex44_HT_C1_HB2/solve/1	core	verified	optimized · mods	z y x	0	0.159	0.252	cold	0.960	0.17
267	finiteAx12_6_190_7_0__0_2_1_1_1_1_0_1_0/symbolic	symbolic	verified	monotone · mods	t s x4 x7 x5 x3 x2 x1 · H=x4	130	0.173	0.0499	cold	1.51	0.11
268	finiteB_6_343_7_0__2_1_1_0_1_0_1_0_1/symbolic	symbolic	verified	monotone · mods	t s x4 x8 x6 x2 x1 x0 · H=x4	2,249	0.176	0.252	cold	1.54	0.11
269	test_elliptic2l_physical/order/1	core	verified	optimized · mods	x4 x6 x3 x1 x0 x5 x2 · H=x4	1,196	0.182	0.293	cold	2.97	0.061
270	test_elliptic2l_physical/order/2	core	verified	optimized · mods	x4 x6 x3 x1 x0 x5 x2 · H=x4	1,196	0.188	0.292	cold	2.57	0.073
271	finiteA_5_286_6_0__0_2_1_1_1_0_0_0_1/symbolic	symbolic	verified	optimized · mods	s t x3 x2 x1 x4 x8	3,709	0.213	0.333	cold	4.34	0.049
272	finiteD_6_373_7_0__2_0_1_0_1_1_1_0_1/symbolic	symbolic	verified	monotone · mods	s x4 x8 x6 x5 x2 x0 · H=x4	3,096	0.219	0.292	cold	timeout	—
273	finiteC_5_158_6_0__0_2_1_1_1_0_0_1_0/symbolic	symbolic	verified	optimized · mods	t s x3 x2 x1 x4 x7	2,197	0.219	0.374	cold	3.00	0.073
274	finiteA_6_190_7_0__0_2_1_1_1_1_0_1_0/symbolic	symbolic	verified	monotone · mods	s t x4 x7 x5 x3 x2 x1 · H=x4	130	0.243	0.0695	cold	1.97	0.12
275	finiteAx12_5_286_6_0__0_2_1_1_1_0_0_0_1/symbolic	symbolic	verified	optimized · mods	t s x3 x2 x1 x4 x8	5,489	0.293	0.496	cold	5.98	0.049
276	test_elliptic2l_physical/solve/3	core	verified	optimized · mods	x4 x0 x1 x3 x6 x5 x2 · H=x4	1,193	0.307	0.556	cold	0.780	0.39
277	test_massless_planar_elliptic_symb/order/1	core	verified	local · given	ss1 ss2 ss4 ss5 ss12 ss23 ss34 ss56 ss61 ss45 ss123 ss234 ss345 x1 x6 x2 x3 x4 x5 x0 · H=x1	12	0.318	0.0290	cold	timeout	—
278	finiteAx12_5_376_6_0__0_0_0_2_1_1_1_0_1/symbolic	symbolic	verified	monotone · mods	t s x4 x6 x5 x3 x8 · H=x4	1,744	0.330	0.273	cold	25.2	0.013
279	test_elliptic2l_physical/solve/4	core	verified	optimized · mods	x4 x0 x1 x3 x6 x5 x2 · H=x4	3,254	0.330	0.616	cold	0.760	0.43
280	test_melih_box/solve/3	core	verified	optimized · mods	x4 x6 x5 x3 x7 x2 x1 · H=x4	3	0.354	0.0296	cold	0.550	0.64
281	finiteA_5_301_6_0__2_0_1_1_0_1_0_0_1/symbolic	symbolic	verified	monotone · mods	s t x0 x3 x2 x5 x8 · H=x0	7,929	0.443	0.941	cold	6.63	0.067
282	finiteAx12_5_301_6_0__2_0_1_1_0_1_0_0_1/symbolic	symbolic	verified	monotone · mods	t s x0 x3 x2 x5 x8 · H=x0	5,462	0.498	0.657	cold	6.20	0.080
283	test_melih_box/order/1	core	verified	optimized · mods	x4 x6 x5 x3 x7 x2 x1 · H=x4	1	0.681	0.0294	cold	0.580	1.2
284	finiteAx12_5_171_6_0__2_1_0_1_0_1_0_1_0/symbolic	symbolic	verified	monotone · mods	t s x0 x7 x5 x3 x1 · H=x0	15,730	0.809	1.65	cold	1.57	0.52
285	finiteA_5_376_6_0__0_0_0_2_1_1_1_0_1/symbolic	symbolic	verified	monotone · mods	s t x4 x6 x5 x3 x8 · H=x4	12,357	0.847	2.00	cold	30.3	0.028
286	finiteAx12_5_31_6_0__2_1_1_1_1_0_0_0_0/symbolic	symbolic	verified	monotone · mods	t s x4 x3 x2 x1 x0 · H=x4	35,354	1.71	4.00	cold	5.71	0.30
287	finiteAx12_6_159_7_0__2_1_1_1_1_0_0_1_0/symbolic	symbolic	verified	monotone · mods	t s x4 x3 x2 x1 x0 x7 · H=x4	40,697†	1.99	4.42	cold	timeout	—
288	test_ex_e5/order/1	core	verified	optimized · mods	p1s x4 x5 x3 x2 x1 x0 · H=x4	0	2.28	5.80	cold	5.05	0.45
289	test_ex_e5/solve/3	core	verified	optimized · mods	p1s x4 x5 x3 x2 x1 x0 · H=x4	11,968†	2.44	5.86	cold	5.75	0.42
290	finiteA_5_171_6_0__2_1_0_1_0_1_0_1_0/symbolic	symbolic	verified	monotone · mods	s t x0 x7 x5 x3 x1 · H=x0	15,730	3.24	4.24	cold	1.51	2.1
291	test_ex_e5/solve/4	core	verified	optimized · mods	p1s x4 x5 x3 x2 x1 x0 · H=x4	16,232	3.42	9.94	cold	6.73	0.51
292	finiteA_5_31_6_0__2_1_1_1_1_0_0_0_0/symbolic	symbolic	verified	monotone · mods	s t x4 x3 x2 x1 x0 · H=x4	74,351	3.49	8.47	cold	4.36	0.80
293	test_ex_e5/solve/5	core	verified	optimized · mods	p1s x4 x5 x3 x2 x1 x0 · H=x4	54,026	3.91	10.4	cold	10.6	0.37
294	finiteA_6_159_7_0__2_1_1_1_1_0_0_1_0/symbolic	symbolic	verified	monotone · mods	s t x4 x3 x2 x1 x0 x7 · H=x4	85,868†	4.16	9.42	cold	timeout	—
295	finiteC_6_438_7_0__0_2_1_0_1_1_0_1_1/symbolic	symbolic	verified	monotone · mods	t s x4 x5 x2 x1 x8 x7 · H=x4	115,955†	8.31	19.4	cold	37.8	0.22
296	test_massless_planar_elliptic/solve/3	core	verified	baseline · given	x4 x3 x2 x6 x5 x1 x0	138,678	8.58	22.8	cold	17.6	0.49
297	test_massless_planar_elliptic/order/1	core	verified	baseline · given	x4 x3 x2 x6 x5 x1 x0	131,072	8.65	22.4	cold	19.5	0.44
298	test_gluza_ex30/solve/3	core	verified	bilinear_positive/local · given	x1 x7 x2 x6 x5 x4 x0	80,980	20.7	15.8	cold	timeout	—
299	finiteC_5_31_6_0__2_1_1_1_1_0_0_0_0/symbolic	symbolic	verified	monotone · mods	t s x4 x3 x2 x1 x0 · H=x4	271,030	21.2	51.3	cold	2.48	8.5
300	finiteC_6_95_7_0__2_1_1_1_1_1_0_1_0/symbolic	symbolic	verified	monotone · mods	t s x4 x3 x2 x1 x0 x6 · H=x4	276,039†	22.8	52.6	cold	timeout	—
301	finiteCx12_6_438_7_0__0_2_1_0_1_1_0_1_1/symbolic	symbolic	verified	monotone · mods	s t x4 x5 x2 x1 x8 x7 · H=x4	115,955†	22.9	37.1	cold	58.6	0.39

#	Problem	Scope	Status	Method · ordering	Order (H = anchor)	Cells	Solve s	Verify s	Cache	Native s	Ratio
302	hanging_test_ex38/solve/1	core	verified	optimized · mods	t z y x	124,744†	28.2	59.7	cold	18.8	1.5
303	test_elliptic2l_physical_symb/order/1	core	verified	monotone · given	pp4 t s x6 x2 x3 x1 x0 x5 x4 · H=x6	269,061	29.0	71.2	cold	timeout	—
304	finiteA_6_317_7_0__2_0_1_1_1_1_0_0_1/symbolic	symbolic	**unresolved**	local · given	s t x4 x0 x2 x3 x5 x8 · H=x4	—	> 3600[w]	—	warm[w]	timeout	—
305	finiteA_6_318_7_0__0_2_1_1_1_1_0_0_1/symbolic	symbolic	**unresolved**	local · given	s t x4 x1 x2 x3 x5 x8 · H=x4	—	> 3600[w]	—	warm[w]	timeout	—
306	finiteA_6_377_10_0__5_0_0_1_1_1_1_0_1/symbolic	symbolic	**unresolved**	local · given	s t x4 x0 x3 x5 x6 x8 · H=x4	—	> 3600[w]	—	warm[w]	timeout	—
307	finiteA_6_414_10_0__0_5_1_1_1_0_0_1_1/symbolic	symbolic	**unresolved**	local · mcts	s t x8 x1 x2 x3 x4 x7 · H=x8	—	> 3600[w]	—	warm[w]	timeout	—
308	finiteA_7_381_10_0__4_0_1_1_1_1_1_0_1/symbolic	symbolic	**unresolved**	local · mcts	s t x8 x0 x2 x3 x4 x5 x6 · H=x8	—	> 3600[w]	—	warm[w]	timeout	—
309	finiteA_7_446_11_0__0_5_1_1_1_1_0_1_1/symbolic	symbolic	**unresolved**	local · given	s t x5 x1 x2 x3 x4 x7 x8 · H=x5	—	> 3600[w]	—	warm[w]	timeout	—
310	finiteAx12_6_317_7_0__2_0_1_1_1_1_0_0_1/symbolic	symbolic	**unresolved**	local · mcts	s t x8 x0 x2 x3 x4 x5 · H=x8	—	> 3600[w]	—	warm[w]	timeout	—
311	finiteAx12_6_318_7_0__0_2_1_1_1_1_0_0_1/symbolic	symbolic	**unresolved**	local · mcts	s t x8 x1 x2 x3 x4 x5 · H=x8	—	> 3600[w]	—	warm[w]	timeout	—
312	finiteAx12_6_377_10_0__5_0_0_1_1_1_1_0_1/symbolic	symbolic	**unresolved**	local · mcts	s t x8 x0 x3 x4 x5 x6 · H=x8	—	> 3604[w]	—	warm[w]	timeout	—
313	finiteAx12_6_414_10_0__0_5_1_1_1_0_0_1_1/symbolic	symbolic	**unresolved**	local · mcts	s t x8 x1 x2 x3 x4 x7 · H=x8	—	> 3600[w]	—	warm[w]	timeout	—
314	finiteAx12_7_381_10_0__4_0_1_1_1_1_1_0_1/symbolic	symbolic	**unresolved**	local · mcts	s t x8 x0 x2 x3 x4 x5 x6 · H=x8	—	> 3600[w]	—	warm[w]	timeout	—
315	finiteAx12_7_446_11_0__0_5_1_1_1_1_0_1_1/symbolic	symbolic	**unresolved**	local · mcts	s t x8 x1 x2 x3 x4 x5 x7 · H=x8	—	> 3600[w]	—	warm[w]	timeout	—
316	finiteB_7_471_13_0__7_1_1_0_1_0_1_1_1/symbolic	symbolic	**unresolved**	local · mcts	s t x0 x6 x7 x2 x1 x4 x8 · H=x0	—	> 3600[w]	—	warm[w]	timeout	—
317	finiteBx12_7_471_13_0__7_1_1_0_1_0_1_1_1/symbolic	symbolic	**unresolved**	local · mcts	s t x0 x1 x2 x4 x6 x7 x8 · H=x0	—	> 3600[w]	—	warm[w]	timeout	—
318	finiteC_6_215_11_0__6_1_1_0_1_0_1_1_0/symbolic	symbolic	**unresolved**	local · mcts	s t x7 x0 x1 x2 x4 x6 · H=x7	—	> 3600[w]	—	warm[w]	timeout	—
319	finiteC_6_231_7_0__2_1_1_0_0_1_1_1_0/symbolic	symbolic	**unresolved**	local · mcts	s t x7 x0 x1 x2 x5 x6 · H=x7	—	> 3600[w]	—	warm[w]	timeout	—
320	finiteC_6_470_10_0__0_5_1_0_1_0_1_1_1/symbolic	symbolic	**unresolved**	local · mcts	s t x7 x1 x2 x4 x6 x8 · H=x7	—	> 3600[w]	—	warm[w]	timeout	—
321	finiteC_6_476_10_0__0_0_5_1_1_0_1_1_1/symbolic	symbolic	**unresolved**	local · mcts	s t x7 x2 x3 x4 x6 x8 · H=x7	—	> 3600[w]	—	warm[w]	timeout	—
322	finiteC_7_247_10_0__4_1_1_0_1_1_1_1_0/symbolic	symbolic	**unresolved**	local · mcts	s t x7 x0 x1 x2 x4 x5 x6 · H=x7	—	> 3600[w]	—	warm[w]	timeout	—
323	finiteC_7_478_10_0__0_4_1_1_1_0_1_1_1/symbolic	symbolic	**unresolved**	local · mcts	s t x7 x1 x2 x3 x4 x6 x8 · H=x7	—	> 3600[w]	—	warm[w]	timeout	—
324	finiteCx12_6_215_11_0__6_1_1_0_1_0_1_1_0/symbolic	symbolic	**unresolved**	local · mcts	s t x7 x0 x1 x2 x4 x6 · H=x7	—	> 3600[w]	—	warm[w]	timeout	—
325	finiteCx12_6_231_7_0__2_1_1_0_0_1_1_1_0/symbolic	symbolic	**unresolved**	local · mcts	s t x7 x0 x1 x2 x5 x6 · H=x7	—	> 3600[w]	—	warm[w]	timeout	—
326	finiteCx12_6_476_10_0__0_0_5_1_1_0_1_1_1/symbolic	symbolic	**unresolved**	local · mcts	s t x7 x2 x3 x4 x6 x8 · H=x7	—	> 3600[w]	—	warm[w]	timeout	—
327	finiteCx12_7_247_10_0__4_1_1_0_1_1_1_1_0/symbolic	symbolic	**unresolved**	local · mcts	s t x7 x0 x1 x2 x4 x5 x6 · H=x7	—	> 3600[w]	—	warm[w]	timeout	—
328	finiteCx12_7_478_10_0__0_4_1_1_1_0_1_1_1/symbolic	symbolic	**unresolved**	local · mcts	— · H=x7	—	> 3600[h]	—	hist.[h]	timeout	—
329	finiteD_7_251_12_0__6_1_0_1_1_1_1_1_0/symbolic	symbolic	**unresolved**	local · mcts	— · H=x0	—	> 3600[h]	—	hist.[h]	timeout	—
330	finiteD_7_375_13_0__7_1_1_0_1_1_1_0_1/symbolic	symbolic	**unresolved**	local · mcts	— · H=x0	—	> 3600[h]	—	hist.[h]	timeout	—
331	test_gluza_ex33/solve/3	core	**unresolved**	local · mcts	— · H=x3	—	> 7200[h]	—	hist.[h]	timeout	—

Cache tags. *cold*: measured in this report's run with the report binary, in a fresh process with a fresh run directory and `TMPDIR` and no persistent checkpoints (no algebra cache shared with any other run); every verified row is cold. The other tags occur only on unresolved rows, which this run did not re-attempt: their **Solve s** column shows the longest earlier attempt, a lower bound and not a solve time. [h] *hist.*: the 2026-10-10 one-hour campaign (Gluza33: 7,200 s), recorded in `benchmark/data.json`; [w] *warm*: a 2026-10-12 one-hour attempt with improved local-engine recipes and a warm shared operation cache (binary `c8f2c62f`, integration `4dbe440`); 4 of these launches found their input withdrawn and are ignored, so those rows fall back to [h]. For `mcts` recipes without a fixed order, the order shown is the one the 2026-10-12 attempt was working on (— where none was recorded). **Native s** is the sum over sign cases of the native GCAD process time (GNU `time` elapsed); *timeout* means at least one sign case hit the 300 s cap. Each attempt's status, wall time, peak memory and binary are in `report-data.json` (`rows[].attempts`).

1.2 Opt-in projective contract (separate table, not counted in the strict total)

These rows use the user-approved contract extension `solver.projective_sections = true` (Section 2, item X1): a cell's upper boundary may be a positive root that escapes to +∞ inside the cell. They are reported separately and **do not change the strict count of 303 / 331**.

Problem (contract)	Engine (base)	Order (H = base anchor)	Cells	Solve s	Verify s	Cache	Same problem, strict contract	Native s	Sanity check
positive Gluza33 (test_gluza_ex33/solve/3) **opt-in contract** `projective_sections = true`	bilinear_general (optimized)	x4 x7 x2 x1 x0 x6 x3 x5 · H=x4	2,372	0.38	0.54	cold	unresolved	timeout	4,744 fresh points in 2,372 cells, 0 failures (checked on binary `4f212724`'s result, whose cells are identical to this run's)
positive Gluza30 (test_gluza_ex30/solve/3) **opt-in contract** `projective_sections = true`	bilinear_general (optimized)	x1 x7 x2 x6 x5 x4 x0	1	0.06	0.03	cold	80,980 cells, 20.68 s	timeout	—

Projective rows: same cold protocol, binary and repeats as the strict table (`benchmark/campaign/projective/`). The sanity check (seed 20261010, two fresh rational points per cell, original TOML expressions evaluated exactly) is outside all timings and supplies no proof credit.

2 Improvements over the original GCAD

The reference is Vitaly Magerya's GCAD (a Python/SymPy front end over FLINT). It implements Strzeboński's 2000 *generic* CAD: generic projection (leading coefficients, discriminants and pairwise resultants of a square-free basis), recursive lifting with exact real-root isolation, the MODS/SOTD/T1 ordering heuristics and an aggressive cell merge. Everything listed below goes beyond that. The list is meant to be exhaustive: every engine, certificate family, projection or kernel change, local-engine mechanism, ordering method, cache, verifier feature, supervision feature, interface, upstream fix and contract extension that exists in the code, plus the investigated ideas that were not adopted.

Vocabulary. A CAD splits the solution set into *cells*. *Projection* eliminates variables one at a time and collects the polynomials (resultants, discriminants, leading coefficients) whose zeros may change the picture; *lifting* goes back up, picking rational *sample* points and isolating real roots above them. The *order* fixes which variable is eliminated first. A *certificate* is an exact, replayable proof (e.g. that a polynomial never vanishes on the domain) that lets a polynomial be dropped. A *guard* is a condition that must hold for a local shortcut to be valid. The *anchor* of a homogeneous problem is the positive variable that is fixed to 1. *Generic* means correct outside an explicit finite list of lower-dimensional exceptional sets. An *engine* is a strategy that turns the problem into one or more smaller CAD problems and rebuilds the original cells with a proof. A *stack* is the list of cells above one cell of the lower-dimensional decomposition: the real roots of the relevant polynomials over that cell cut its cylinder into *sections* (the root graphs themselves) and *sectors* (the open slices between consecutive roots). A polynomial is *delineable* over a cell when its real roots keep their number and order everywhere above that cell, so they can serve as stack boundaries. A *seam* is a boundary between two neighbouring cells on which the target keeps the same sign; GCAD's merge may remove it, symGCAD keeps it (V4).

Tags. default-on active without any setting (for a mechanism of an opt-in engine, e.g. the local engine: active whenever that engine runs); opt-in enabled by an option, engine or ordering choice (the option name is given); optional patch optional dependency patch, never in the default build; diagnostic measurement or audit tool, not part of a solve; contract extension changes what a result means and must be requested explicitly; NEW 2026-10-12 written in the 2026-10-12 session.

Reading the numbers. Unless a line says otherwise, an effect is one measured case or operand: it shows what the change does, not a corpus-wide speedup (corpus-wide timings are in Section 1, kernel-level comparisons with FLINT in Section 3). "Timeout" results never count as solved. All reductions keep the exact default contract: every cell is proved, every interior `F=0` sheet and same-sign seam stays a boundary. The only exception is the opt-in contract extension X1, which must be switched on explicitly and is never counted in the strict totals.

Where the strict completions come from. The 303 groups verified in Section 1 were solved by these engines (P1, P2, E1–E7, LP1) and ordering policies (GCAD's own `mods` heuristic, or an explicit `given` order, O1); every one was re-checked by the independent verifier (V3):

engine (strict, verified)	groups
`monotone`	218
`optimized`	79
`linear_negative`	2
`baseline`	2
`local`	1
`bilinear_positive`	1

ordering policy	groups
`mods`	294
`given`	9

The `monotone` rows include the unsplit problems handled by its sign filter (E5) and problems combined with the homogeneous reduction (E1); their endpoint bases are solved by the optimized engine (P2). The recipes are those of the 2026-10-10 audit (`benchmark/campaign/symgcad_inputs/`); none of them uses the newer opt-in `kinematic_portfolio` ordering or `bilinear_general` engine.

2.1 Exact algebra and kernels

- A1 **Rust and Symbolica exact backend.** GCAD drives FLINT through Python and SymPy; symGCAD does all polynomial arithmetic, GCDs, factorization and resultants in Rust on Symbolica, inside single-threaded supervised worker processes. No floating-point number enters any proof. *Effect:* in the matched parity audit (release `9025e71`, identical orders, plain `baseline` projection), symGCAD reproduced the raw cell counts of all 483 sign-expanded problems GCAD completes; projection + lifting + merging took a geometric mean of 0.1466 of GCAD's time (6.82× faster), whole processes 0.0620 (16.1×). The current end-to-end comparison is in Section 1. default-on
- A2 **Sign-preserving exact input normalization.** Every inequality becomes "this polynomial is positive"; denominators and rational content are cleared without ever flipping a sign. The parser tells an empty problem (`false`, a strict zero constraint) apart from a legitimate identically zero split, and it refuses exponent overflow, undeclared symbols and approximate (floating-point) kinematics instead of guessing. default-on
- A3 **Full variable maps.** Every polynomial keeps its complete ordered variable list, including variables it does not depend on. A cached result computed under one ordering or coordinate chart can therefore never be reused for another one that merely looks the same. default-on
- A4 **Deterministic resultants.** Symbolica's automatic resultant dispatcher may accept a modular (CRT) answer after random-prime checks, which is not a proof. Production and verification use the deterministic integer Lazard–Ducos subresultant algorithm (plus the exact kernels A6–A9); the dispatcher is confined to profiling, and caches built with it were invalidated. *Effect:* on large captured operands Symbolica's Ducos runs at 0.74–0.88× the time of FLINT's Ducos (Section 3). default-on
- A5 **Closed-form low-degree discriminants.** The discriminant of a quadratic, cubic or quartic is evaluated from its exact invariant formula instead of the generic `Res(p, p')`, which expands a large Sylvester determinant. *Effect:* captured quartic (642 ⟶ 69,474 terms): 2.71 s / 48 MiB against 29.77 s / 472 MiB (10.98× time, 9.83× memory); on captured kinematic discriminants the production path is 6–18× faster than FLINT 3.6 (Section 3). default-on
- A6 **Exact evaluation/interpolation resultants.** When a resultant or discriminant depends on one remaining parameter, it is computed at many integer values of that parameter (each a plain integer resultant) and rebuilt by exact interpolation; a proved degree bound says how many values suffice, so the answer is exact, not probabilistic. *Effect:* captured symbolic A446 pair (degrees 23 and 17, 588/387 terms): 5.888 s with 872 images; every Ducos variant exceeded 60 s. default-on
- A7 **Assignment degree bound and size-derived admission.** The number of interpolation points is the smaller of the closed-form Sylvester degree bounds and an exact assignment (Hungarian algorithm) bound on the Sylvester matrix, which is often much tighter. Plans outside the historical envelope (closed-form bound ≤ 1,024), up to 65,536, are admitted only if an a-priori bit bound on every image fits the caps, so the kernel cannot run out of budget after it has been chosen. *Effect:* degree-1,828 discriminant: 27.80 s (old multivariate-image interpolation 52.31 s, old default path > 660 s); degree-2,220: 58.83 s against 106.79 s. NEW 2026-10-12 default-on
- A8 **Exact integer images with a zero-image guard.** Each interpolation image is an exact integer resultant; an exact gcd test first decides whether it is zero, because Symbolica 3.0.1's Lazard–Ducos panics on a vanishing subresultant (U3). The same guard protects the full Ducos path. *Effect:* a pair on which the previous release crashed now solves (7 cells) and verifies. NEW 2026-10-12 default-on
- A9 **Binary (Kronecker) segmentation.** For two-parameter resultants and discriminants, the parameters are packed into one integer variable by substituting large powers of two under proved degree and coefficient bounds; one univariate integer resultant then replaces a multivariate computation, and the result is unpacked with complete signed checks. *Effect:* authentic A317/A318 resultants of 9,835/18,939 terms in 38.16/20.03 s; forced on a 56-degree-product discriminant 2.17 s against Ducos 7.80 s; on the captured large resultants it is 3–16× faster than FLINT 3.6's best kernel (Section 3). Default since 2026-10-12 (`binary_resultants = false` opts out). NEW 2026-10-12 default-on
- A10 **Second-route spot check.** Because A6–A9 are new code paths, every interpolated or segmented result is re-evaluated at one extra full-degree point through a different Symbolica route (substitution, then Brown's resultant); a disagreement is a hard failure. This runs in the solver and again in the verifier. *Effect:* cost within noise (27.86 s against 27.87 s). NEW 2026-10-12 default-on
- A11 **Sum-variable factor preconditioning.** A polynomial that depends on two variables only through their sum is factored as a polynomial in that sum (one variable fewer) and mapped back; the inverse substitution and the factor product are re-checked. *Effect:* a stalled A446 factorization (native > 60 s) took 0.003–0.025 s. default-on
- A12 **Affine factorization preconditioning.** Multivariate factorization can stall when the polynomial is degenerate at the evaluation point the factorizer picks. A bounded exact integer translation first moves the polynomial to a regular fiber; the factors are mapped back and their signed product is rebuilt to prove that nothing was lost. *Effect:* 224-term Gluza33 factor 1.63 s, where the native call timed out at 30.02 s; one operand showed about 1.42× overhead, so it stays opt-in. opt-in (`affine_factorization`)
- A13 **Support-preserving diagonal scaling.** Tried first inside the affine policy: multiplying variables by nonzero constants reaches a regular fiber without creating new monomials (a translation can blow up the

term count). *Effect:* a 1,283-term factor on which translation stalled for 30 s factors in 0.048 s. opt-in (inside `affine_factorization`)

- A14 **Bounded exact shear.** Third strategy of the affine policy: the substitution `x -> x + c·y` for a small integer c, with an exact inverse, for polynomials on which translation and scaling fail. *Effect:* captured C470 obstruction 0.088 s; the original C470 continuation then saved 56,671 more exact operations (still incomplete). opt-in (inside `affine_factorization`)
- A15 **Primitive parameter-linear irreducibility shortcut.** A polynomial that is linear in one variable and whose coefficients share no common factor is irreducible (Gauss's lemma), so the factorizer is not called at all. *Effect:* 4.840 ⟶ 0.120 ms and 1.463 ⟶ 0.079 ms on two core inputs. default-on

2.2 Root isolation and lifting

- L1 **Algebraic cell boundaries, re-isolated at every prefix.** A boundary is stored as "the k-th root of polynomial p", not as a number. At each new rational prefix its roots are isolated exactly again; coincident roots are separated by exact gcds and interval refinement under a work bound. The proof records the boundaries, so the verifier can recheck every one. default-on
- L2 **Open-interval Sturm counting.** Real roots on open or unbounded intervals are counted with signed primitive pseudo-remainder sequences, which keep coefficients small (Symbolica alone counts only on `(0, ∞)`). *Effect:* degree-12 A381 guard 3.184 ⟶ 0.602 ms, largest coefficient 2,281 ⟶ 555 bits. default-on
- L3 **Integer sign evaluation.** The sign of a polynomial at a rational point is computed by clearing the denominator and running Horner's rule on integers, instead of rational arithmetic with a gcd at every step. *Effect:* 2.64–11.01× in a microbenchmark. default-on
- L4 **Integer prefix evaluation.** The same idea for substituting a whole rational prefix into a multivariate polynomial: exact integer numerators with one common denominator. *Effect:* guard batches 3.79× (A446) and 5.33× (A381); the matched whole-solver control moved < 1% (43.68 ⟶ 43.30 s). default-on
- L5 **Single-pass batched specialization.** All coordinates of a prefix are substituted in one pass that shares the powers of each coordinate, instead of one variable after another. *Effect:* 2,000 prefixes of `ex_e5/2`: 0.401 ⟶ 0.252 s; ex38: 0.079 ⟶ 0.052 s. default-on
- L6 **Deferred GCDs and borrowed root signs.** GCDs that are often unnecessary are postponed until a collision actually has to be resolved, and signs at a root reuse data already computed during isolation. *Effect:* improved the three cases where symGCAD was slowest relative to GCAD in the parity audit; their time ratios remain 2.09, 1.61 and 2.75. default-on
- L7 **Primitive shared root fibers.** All root intervals of one specialized polynomial share one immutable defining polynomial; retained rational fibers are made primitive integer polynomials; affine prefix guards skip power tables. Less copying and smaller numbers during lifting. *Effect:* A414 fixed-work traversal 7.748 ⟶ 6.917 s (−10.7%; a first pair was 5.3% slower). default-on
- L8 **Ordered root union.** When the roots of many polynomials have to be merged into one sorted list of boundaries, a priority queue merges them, with exact equality and bisection checks so that equal or extremely close roots land in the right order. No speedup established; it is the correctness backbone of LP3. default-on
- L9 **Positive and descending root selectors.** A boundary may be "the k-th positive root" (`positive`) or "the k-th root counted from the largest" (`real_descending`). On a positive domain this keeps its meaning when irrelevant negative roots collide or disappear. Only certified rules produce such selectors, and the verifier rechecks them. default-on (produced only by certified rules)

2.3 Projection operators and certified reductions

- P1 **Faithful generic projection.** The `baseline` engine reproduces Strzeboński's projection and lifting exactly; it is the reference for A1 and the fallback for every engine. default-on (`engine = "baseline"`)
- P2 **Optimized projection engine.** The same projection, but every exact operation is cached under a canonical key and reused across levels, ordering trials and restarts (K1–K3), and generated factors may be removed with certificates (P3). Most strict corpus rows run it, directly or as a base engine. opt-in (`engine = "optimized"`)
- P3 **Certified factor removal.** A generated projection factor is dropped only with an exact certificate (C1–C5) that it is strictly nonzero on the domain; nonnegativity is not enough, because a zero sheet inside the domain would be lost. Original constraints and requested splits stay protected; `input_positivity` extends the rule to input factors. *Effect:* fixed A446: 1.326 s with it, timeout at 60 s without. opt-in (`positivity`, `input_positivity`)
- P4 **Operation pruning.** Tries a nonvanishing proof before factoring a whole projection operation, so a provably irrelevant resultant is never expanded. Off by default because a failed proof costs time. opt-in (`prune_operations`)
- P5 **Strict-Bézout pair omission in global projection.** A supplied identity `p*L + q*M = k*S` with `k > 0`, where `S = H^2` with `H` certified nonzero or `S` certified positive, proves that `p` and `q` have no common zero in the box, so `Res(p, q)` is skipped. `p`, `q`, their leading coefficients and discriminants stay, face pairs are never skipped, and every omission is replayed by the verifier. *Effect:* Gluza33 piece `R--`, third level: 1,033 ⟶ 661 polynomials and 228,998 ⟶ 112,759 terms (−36%, −51%), reached at 122.4 s instead of 164.2 s. NEW 2026-10-12 opt-in (`projection_strict_bezout_hints`)

2.4 Positivity and domain certificates

- C1 **Exact nonvanishing certificate portfolio.** A certificate proves that a polynomial is strictly positive (or negative) on the whole box of interest, so it can never be a cell boundary there. Cheap tests come first: coefficient signs, positive coordinate shifts, Pólya and Bernstein expansions, univariate Sturm counts on open and unbounded domains and exact rules for quadratics, each under a work bound. Every success is serialized into the proof and replayed by the verifier on the original domain. default-on (used when `positivity` is set)
- C2 **Parameterized quadratics.** A quadratic whose discriminant is proved negative on the whole domain (with a positive leading coefficient) is positive there. default-on (inside C1)
- C3 **Square plus positive residual.** Writes a polynomial as an exact square (of a monomial or an affine form, also after a homogeneous scale is fixed) plus a strictly positive residual, with oriented bounds. default-on (inside C1)
- C4 **Weighted-square certificates.** Sums of binomial squares with positive monomial weights plus a positive residual, for polynomials that are positive but have mixed-sign coefficients. *Effect:* "positive endpoint" case: 20 ⟶ 1 cell, 37.2 ⟶ 9.3 ms solver time. opt-in (`weighted_square_positivity`)
- C5 **Positive-semidefinite quadratic components.** Exact rational LDL elimination of quadratic subforms plus a strictly positive residual; no floating-point eigenvalues. *Effect:* full quadratic constant 8 ⟶ 1 cell (18.3 ⟶ 7.6 ms); local linear split 12 ⟶ 2 cells, with the interior zero sheet kept. opt-in (`quadratic_component_positivity`)
- C6 **Caller-chosen certificate budgets.** Library callers may raise the limits above the defaults of 4,096 Bernstein coefficients and 10^6 work units (symGCAD PR #2 allowed up to 262,144 and 10^9; C9 raises these ceilings); the chosen limits are written into the proof and used again by the verifier. opt-in (library `CertificateOptions`)
- C7 **Early abandonment from exact samples.** An exact zero or a sign change found at a sample point ends a hopeless certificate attempt immediately. Samples only ever stop a search; they never prove anything. default-on
- C8 **Certified local guards.** In the local engine, a certificate can replace a generated guard polynomial; the certificate's own premises become protected obligations of the ancestor cell. *Effect:* fixed A446 local: 8 certificates replaced 436 guard occurrences; 35 cells in 0.427 s, where the unreduced run timed out at 60 s. opt-in (`positivity` with `engine = "local"`)
- C9 **Larger explicit certificate allowances.** The C6 ceilings rise to 2,097,152 Bernstein coefficients and 6.4 × 10^{10} work units (defaults unchanged). The tensor size is checked before allocation, and the work allowance is validated and debited without overflow. *Effect:* regression tensors of 263,169 (two variables) and 1,115,370 coefficients (eight variables), refused under the earlier ceilings, now give exact certificates that replay from their saved options; lowered saved limits are rejected. NEW 2026-10-12 opt-in (library `CertificateOptions`)

2.5 Composite engines

- E1 **Homogeneous positive-scale reduction.** Feynman-parameter polynomials are homogeneous in a block of positive variables: scaling the whole block does not change any sign. One proved-positive anchor coordinate is therefore fixed to 1, the smaller problem is solved, and the original cells are rebuilt exactly. *Effect:* fixed A446 2.409 ⟶ 1.326 s, fixed Ax12 1.032 ⟶ 1.445 s (mixed, hence opt-in). opt-in (`homogeneous_block`, `homogeneous_anchor`)
- E2 **Monotone engine.** If certificates prove that `F` strictly increases along a positive axis, the solution set along that axis is one interval ending at a root: the axis is removed, the endpoint problem is solved, and every root sector is restored with its proof. *Effect:* fixed A446/Ax12: 35 ⟶ 8 cells; A446 1.326 ⟶ 0.825 s. opt-in (`engine = "monotone"`, base engine `monotone_base_engine`)

- E3 **Conditional quadratic peel.** When `F` is quadratic in an axis, positive-multiplier certificates justify consecutive endpoint reductions on each side of the vertex. *Effect:* fixed A381 and Ax381: 836 verified cells each; in this report's cold runs (Section 1) 0.067 s solve, 0.07 s verify and 0.065 s solve, 0.069 s verify (earlier matched control, build `0a95bc2`, medians of 3: 0.186 and 0.102 s solve); all 6 trials without the rule timed out at 60 s. opt-in (inside `monotone`)
- E4 **Negative-target mode.** Solves `Q > 0` as the negative side of `F = -Q` through its endpoint inequality, restoring only the interval below the positive root. *Effect:* full original physical elliptic problem (`test_elliptic2l_physical_symb/order/1`): 269,061 cells; this report's cold run 29 s solve, 71.2 s verify (first release run, build `1e29f04`: 33.86 s / 73.09 s). opt-in (`monotone_negative_target`)
- E5 **Sign filter.** For a problem without a requested split, decomposes the full sign arrangement of one non-affine constraint and keeps exactly the required cells, with an exhaustive selection map that the verifier checks. opt-in (inside `monotone`)
- E6 **Linear-negative engine.** A target linear in `z` with `A*z + B < 0` is reduced to `A < 0`, using a guide polynomial whose positivity and square identity `B*C - A*D = k*H^2` are proved; the lower boundary is the largest root of `z*q`. *Effect:* negative Gluza30 (`test_gluza_ex30/order/1`): 1 cell, 0.052 s solve, 0.03 s verify; negative Gluza33 (`test_gluza_ex33/order/1`): 3 cells, 0.049 s solve, 0.03 s verify (this report's cold runs; earlier release medians of 3, build `256527b`: 0.185 s and 0.164 s solve); GCAD timed out at 60 s on both in those earlier controls. opt-in (`engine = "linear_negative"`, `negative_linear_guide`)
- E7 **Bilinear-positive engine.** For `a*x*y + b*x + c*y + d > 0` on the last two positive axes, seven strict certificates against a guide polynomial fix the shape of the two-dimensional fiber; only the base (the remaining variables) is decomposed, over the signs of `a`, `b`, `c`. *Effect:* positive Gluza30: 80,980 verified cells; this report's cold run 20.7 s solve, 15.8 s verify (release controls, build `ff22294`, medians of 3: 22.57 s / 16.85 s); GCAD timed out. Under the opt-in contract X1 the same problem needs 1 cell. opt-in (`engine = "bilinear_positive"`, `bilinear_positive_guide`, `bilinear_positive_base_engine`)
- E8 **General bilinear engine with pieces.** Handles all 16 sign patterns of `a, b, c, d`: ratio premises between them are found from the Symanzik guide by square-free decomposition, a separator `G` satisfies `b*c - a*d = ±m*G^2`, and a planner splits the base into independently solvable pieces plus proved-empty regions (objective `region_weighted`, the default, or `atoms`). The verifier checks the partition by a truth table and verifies every piece. *Effect:* Gluza33: a plan of 4 pieces + 1 empty claim in 0.034 s; a 4-piece synthetic chain solved as 4 separate processes and assembled gave 47 verified cells (pieces CLI test); on a different synthetic input, a 3-parameter quartic chain with the local base engine, `region_weighted` was 4–14% faster than `atoms` (8,142 against 12,015 cells); under the strict contract the four Gluza33 pieces still time out (1,200 s); with X1 the engine completes positive Gluza33. NEW 2026-10-12 opt-in (`engine = "bilinear_general"`, `bilinear_general_guide`, `bilinear_general_plan_objective`)
- E9 **Quadratic ladder.** When a quadratic opens two positive roots below a linear fiber, restores a proved five-sector arrangement over a verified base, keeping both sides of `F = 0`. *Effect:* crossed A446 chamber `t < 4`: 5 verified cells in 0.022 s (debug); the unrestricted problem stays unresolved. opt-in (`engine = "quadratic_ladder"`)
- E10 **Selectable base engines and inner anchors.** Monotone and bilinear wrappers solve their base with `baseline`, `optimized` or `local`, optionally with an inner homogeneous anchor and a block that keeps kinematic parameters outside the scale block. opt-in (`monotone_base_engine`, `bilinear_*_base_engine`, `bilinear_general_base_anchor`; `bilinear_general_base_block` is NEW 2026-10-12)
- E11 **Projection-free peel inspection.** Reports which monotone peels are certifiable for an input without solving anything, to choose recipes cheaply. diagnostic

2.6 Local projection engine

- LP1 **Local projection.** Ordinary CAD projects every pair of polynomials everywhere. The local engine, an adaptation of Strzeboński's local projection, projects only what the sample points it actually visits need: it starts with few cuts, learns the missing ancestor cuts a stack requires and rebuilds the affected stacks. The whole conditional tree is checked by the independent verifier. Discovery order matters, which is what LP12, LP19 and LP22 address. opt-in (`engine = "local"`)
- LP2 **Batched sibling discovery.** Collects new obligations from several sibling stacks before rebuilding their parent, instead of rebuilding after each one; a tentative stack with new cuts is always discarded, only a fully saturated pass produces cells. default-on (`local_batch_siblings = false` restores one-sibling discovery)
- LP3 **Direct first-axis base.** On the first lifting axis, univariate cuts need no pair resultants: the union of their real roots is isolated directly (with L8), and the verifier checks this zero-dimensional base condition. *Effect:* a fixed A381 run avoided 4,150,168 potential first-axis pairs. default-on (`local_base_point_projection`)
- LP4 **Positive-ray Descartes rule.** On a proved-positive axis, zero or one sign variation in the coefficients proves "no positive root" or "exactly one simple positive root", replacing the discriminant. default-on (`local_descartes_one`)
- LP5 **Quadratic positive-ray classifier with cached guards.** Exact sign cases of a quadratic's coefficients decide its positive roots; guard results are cached across stacks. *Effect:* fixed-work A381 debug controls: solver time −15.0% and −14.5% (process wall time mixed). default-on
- LP6 **Leading-term cancellation separation.** Up to two exact cancellations whose coefficients all have one certified sign prove that two polynomials share no positive root, so their pair resultant is skipped; inconclusive cases keep it. default-on (`local_pair_cancellations = 2`)
- LP7 **Adjacent-root pairs.** Projects only the pairs whose roots are adjacent at the sample. *Effect:* no completion gain: all 27 symbolic release attempts timed out. opt-in (`local_adjacent_pairs`)
- LP8 **Descartes witnesses for 2–8 positive roots.** Rational separators with alternating signs, plus coefficient guards, prove that each interval holds exactly one simple positive root, replacing a large discriminant. At most two templates per polynomial keep the guard set finite; the source size budget is 1–65,536 terms. *Effect:* with 8,192 terms, the original Gluza33 passed a 6,938-term degree-16 discriminant and saved 4,554 more operations. opt-in (`local_descartes_witnesses`, `local_descartes_witness_max_terms`)
- LP9 **Interval Descartes covers.** Covers the positive ray by rational intervals with zero or one sign variation each, for fibers where the global Descartes count is not exact. *Effect:* Gluza33 19,942-term degree-24 discriminant (2 roots, 4 variations) bypassed with 49 guards (factored in 9.27 s): +6,422 operations; a 131,072-term aggregate budget gave +7,194; template limit 4 gave +1,094 and +2,397. opt-in (`local_descartes_intervals`, `local_descartes_interval_max_terms`, `..._max_total_terms` (default 65,536, ceiling 262,144), `..._template_limit`)
- LP10 **Adjacent interval coalescing.** Merges neighbouring cover intervals when an exact recount keeps at most one variation, which shrinks the guard set. *Effect:* four-root cover with 97 guards / 109,726 terms built in 1.60 s; the greedy cover needed 121 / 139,654, over budget. default-on (when intervals are enabled)
- LP11 **Positive pair witnesses.** Once both polynomials' positive roots are proved delineable, rational separators between them replace the pair resultant. Sample-dependent, without a termination proof; caused both measured thrash cases (LP22). opt-in (`local_positive_pair_witnesses`)
- LP12 **Pair-witness admission gate and axis budget.** A new separator template is admitted only when the resultant it replaces would be large (a dense term bound above a threshold), and per-axis template counts can be capped; otherwise the resultant, always admissible, is used. *Effect:* Gluza33 base piece `a > 0` with witnesses: 20.4 million nodes in 570 s without finishing → 0.02 s and 178 verified cells at gate 4,096; on 19 of 20 corpus cases the gate behaved like witnesses off. Default 0 keeps the old policy. NEW 2026-10-12 opt-in (`local_pair_witness_min_resultant_terms`, `local_pair_witness_axis_budget`)
- LP13 **Local strict-Bézout hints.** The local counterpart of P5 for chosen cut pairs. *Effect:* positive Gluza30: 49 hints skipped 2,918 resultants (no completion). opt-in (`local_strict_bezout_hints`)
- LP14 **Completed-subtree reuse.** A complete conditional proof below a rational prefix is reused when the same prefix comes back, after its obligations are rechecked; budget 32 MiB, entries of 128–4,096 nodes. *Effect:* C470 at an equal refinement limit 8.40 → 6.96 s with 1,024-node entries; no full-solve gain. default-on (`local_reuse_subtrees`, `local_subtree_cache_mib`, `local_subtree_cache_entry_nodes`)
- LP15 **Learned-cut hints.** Unfinished stacks keep the cuts they learned as hints for the next visit (8 MiB); hints are always re-projected, never trusted. default-on (`local_reuse_learned_cuts`, `local_learned_cut_cache_mib`)
- LP16 **Persistent hint archive.** Saves cut hints across restarts (8 MiB), re-checked on load. *Effect:* no completion gain in matched restarts. opt-in (`local_hint_archive`)
- LP17 **Stable samples.** Keeps a rational sample across refinements when an exact comparison places it strictly inside a new sector, so caches keyed by the sample still hit. opt-in (`local_stable_samples`)
- LP18 **Deferred projection with cut compaction.** Discovers complete children before projecting, then drops learned cuts that no child needs. *Effect:* A414 pilot still timed out at 300 s. opt-in (`local_deferred_projection`)
- LP19 **Retained sibling proofs.** Completed children survive a stack's refinement rounds: cuts only grow at a fixed prefix, so each child moves to the new sector that strictly contains its unchanged sample (checked exactly against the roots). Retained pools share a quarter of the memory limit at 4 KiB per node. *Effect:* Gluza33 piece `Ppm`, 30 min: 12.80 → 1.38 million speculative nodes for the same progress (still incomplete); fixed `B6_469` (witness recipe, gate 4,096) completes only with retention: 112,124 verified cells in 9.91 s. Stays

opt-in because it changes samples and hence cell counts. NEW 2026-10-12 opt-in (`local_retain_siblings`, requires `local_stable_samples`)

- LP20 **First-axis square-free support basis.** Replaces irreducible factorization of bounded generated first-axis guards by checked square-free blocks, whose product identities the verifier replays. *Effect:* 92.06 s against 185.83 s on one A377 block set. opt-in (`local_base_univariate_basis`)
- LP21 **Discovery caps.** `local_max_refinements` (10,000) and `local_max_nodes` (100,000) bound speculative work; exhaustion returns `incomplete`, never a partial answer. default-on
- LP22 **Refinement-thrash diagnostics.** Read-only counters of restarts, cut origins, discarded children and cache-miss causes; a test checks that proofs and counters are identical with it on and off. *Effect:* found 337,116 completed children discarded in 60 s on one Gluza33 piece, which motivated LP12 and LP19. NEW 2026-10-12 diagnostic (`SYMGCAD_LOCAL_THRASH_DIAG=1`)
- LP23 **Obstruction observer.** Records which guard fails at an exact prefix without changing proofs, to locate the next bottleneck. diagnostic

2.7 Ordering and search

- O1 **Extra ordering heuristics.** Brown and sparsity orders join GCAD's MODS, SOTD and T1; `variable_groups` and explicit orders bind every candidate, so parameters stay outside the integration variables. opt-in (`ordering`)
- O2 **Bounded MCTS order search.** Monte-Carlo tree search (UCT) over legal elimination prefixes: seed 0, at most 256 rollouts, search time `min(300 s, 20%)` of the limit, promotion tiers of 1/10/60 s; only a completed exact solve can win. *Effect:* 126/126 cold ablation solves verified, but no end-to-end gain over the cheap heuristics (details in the separate ordering/MCTS companion). opt-in (`ordering = "mcts"`, `search_time_secs`, `max_rollouts`)
- O3 **Search pruning.** Abandons ordering trials whose completed prefixes already show severe term growth. It is a prediction that only saves search time; it never rejects a region and never affects the final solve. default-on (with MCTS; `search_pruning`)
- O4 **Local-progress promotion score.** Ranks timed-out local trials by polynomial inventory and refinement growth, so the most promising order gets the longer budget. *Effect:* reduced positive Gluza30 base: 3/3 verified, median 65.10 s, against 3/3 incomplete at 313.63 s. opt-in (`search_local_progress`)
- O5 **Kinematic portfolio ordering.** For parametric Feynman inputs the cost is driven by the number of cells in the kinematic variables. The portfolio probes the default MODS order, every certified homogeneous anchor and every kinematic order in killable workers, counts the exact open kinematic cells of each, and solves with the fewest (written back as `ordering = "given"`, so solver and verifier are unchanged); decoupled kinematic variables are lifted last. *Effect:* 26 solved cases: 105.88 → 92.97 s total; C531 21.22 → 2.27 s (11,413 → 36 kinematic cells); 4 held-out cases where the default ran out of 5 GiB now solve (5.7–47.6 s); losses `Cx12_438` 32.30 → 59.82 s and 36 tiny cases 2.07 → 7.73 s; closes none of A317, A318, C231. NEW 2026-10-12 opt-in (`ordering = "kinematic_portfolio"`, `portfolio_*` budgets, `portfolio_kinematic_cell_cap` 1,000,000, `portfolio_fiber_samples` 8)
- O6 **Reproducible portfolio budgets.** Fixed per-candidate budgets give the same choice on 21/21 cases on a loaded host, at higher cost (1,317 s on the 26 cases). NEW 2026-10-12 opt-in (`portfolio_fixed_budgets`)
- O7 **Order proposal helpers.** Support-order seeds and short linear-prefix probes; measured, no gain. diagnostic

2.8 Caching and checkpointing

- K1 **Exact operation cache.** Factorizations, resultants, discriminants and completed projection prefixes are shared across stages and ordering trials, so a second order that shares a prefix pays nothing for it (128 MiB). default-on (optimized and local engines; `cache_memory_mib`)
- K2 **Persistent checkpoints.** Atomic, hash-validated on-disk records of every completed exact operation survive interruptions mid-stage, so a long campaign can resume. *Effect:* the final original Gluza33 campaign retained 32,456 operations (4,559 new) for continuation. opt-in (`checkpoint_dir`; automatic with MCTS)
- K3 **Domain-free operation keys.** Pure algebra records are keyed by variable map, axis, backend version and exact input, no longer by the domain, so different region pieces of one problem share algebra; old records are validated and migrated. *Effect:* Gluza33 `R--` after `R+-`: 50 of 127 operations from disk through level 2 (old scheme: 0). NEW 2026-10-12 default-on (with `checkpoint_dir`)
- K4 **Primitive-linear cache bypass.** Trivially irreducible factors (A15) skip cache input/output entirely. *Effect:* 30 polynomials, fresh memory and disk caches: 15.531 → 0.415 ms. default-on
- K5 **Search journal.** MCTS state (8 MiB, 4,096 records) is replayed for the same problem and executable after a restart; replayed claims count as unverified until re-solved. default-on (with MCTS)
- K6 **Untrusted proposal caches.** First-axis support proposals and hint archives are re-factored and re-checked on load; the verifier never reads any cache. opt-in
- K7 **Cache integrity audits.** Inherited bytes, operation keys and output hashes are checked before a continuation reuses work. diagnostic

2.9 Verification and evidence

- V1 **Explicit generic-coverage contract.** A result proves that its cells lie in the solution set `S` and cover `S` up to the zero set of an explicit finite list of nonzero exceptional polynomials, and that the cells are disjoint. GCAD returns cells without such a list. default-on
- V2 **Self-contained JSON proof.** Records the normalized problem, the actual order, the polynomial table, projection levels, rational samples, root-index boundaries, an exhaustive tree of stacks, rejections and accepted cells, and one of four statuses (`complete_generic`, `incomplete`, `invalid_input`, `computation_failure`). default-on
- V3 **Independent verifier.** `symgcad verify` re-normalizes the original input, rebuilds every projection and reduction obligation, checks roots with exact arithmetic and walks every open interval; every engine has its own typed replay; an incomplete result cannot pass. Every verified row of Section 1 passed it in a separate process. default-on
- V4 **Raw cells and seams by default.** No merging unless asked: interior `F = 0` sheets and same-sign seams always remain boundaries (GCAD merges aggressively, and some of its merges cross such seams). default-on
- V5 **Indexed compatibility merge.** GCAD-style merging as a separate output that records which seams were removed. *Effect:* the 131,256-cell `ex38` now completes (median 40.72 s, 4.77 s merging; GCAD 26.50/7.45 s). opt-in (`merge = "aggressive"`)
- V6 **Replay hardening for the new rules.** Strict-Bézout omissions (11 tampered variants rejected), `bilinear_general` evidence (about 40 forgeries rejected), the assignment degree bound (checked through its LP dual), projective plans (X1) and unresolved portfolio inputs are all checked or refused by the verifier. NEW 2026-10-12 default-on
- V7 **Certified comparison of decompositions.** Exact common refinement of two results, across different orders and reductions, to show that two proofs describe the same set. diagnostic
- V8 **Seeded interior sanity check.** Draws fresh rational points in every cell (seed 20261010, two per cell) and evaluates the original TOML expressions exactly; outside all timings and no proof credit. *Effect:* fixed A446: 8 cells, 16 `F` and 112 constraint evaluations, 0 failures, 0.41 s; positive Gluza33 under X1: 4,744 points, 0 failures. diagnostic (`examples/sanity_sample.rs`)
- V9 **Native comparison harness and parity audit.** Frozen GCAD cohort, three paired runs, adaptive native cap `max(300 s, 3 × symGCAD solve)`; documents GCAD merge and assertion defects (e.g. `failing_test_ex43`). diagnostic
- V10 **Signed grid verification of huge algebra.** The A377 degree-32 discriminant candidate (degree 4,058, 3,261 terms) was proved by 4,773 exact evaluation points in 2,093.9 s. diagnostic
- V11 **Original-identity audit.** Every claimed completion is re-checked against the original polynomials, domain, groups and order: 303 of 331 distinct problems (124/124 fixed ZZ, 97/124 symbolic ZZ, 82/83 core), re-confirmed by the cold runs of Section 1. diagnostic

2.10 Robustness and supervision

- R1 **Killable worker processes and limits.** Solves, verifications and probes run in separate process groups with hard wall-time and memory limits (defaults 1 h, 64 GiB, 16 workers; up to 5 h on request, R11) and an optional cell cap, so a blocking native call cannot hang a run and a timeout is reported, not lost. default-on (`[limits]`: `wall_time_secs`, `memory_mib`, `workers`, `max_cells`)
- R2 **Campaign and slow-suite runners.** `symgcad bench` and the resumable one-hour slow queue record wall, CPU and RSS; timeouts stay incomplete. default-on
- R3 **Fast regression suite.** 759 tests in 59 test targets pass after the all-in-main consolidation (0 failed, 2 expensive tests ignored, 350 s summed test time); they include exact rationals and signs, proof mutations, same-sign seams, repeated and nearby roots, degree drops and worker termination. *Effect:* grew from 662 at `81d9ac5` (October 11) to 753 at `9baf91c` with the 2026-10-12 work, then to 759 with the C9 tests and the A381 `mz2=1/2` regression. default-on (`scripts/test.sh fast --locked`)

- R4 **Operand capture and replay.** `SYMGCAD_CAPTURE_DIR` saves the exact operand of the latest uncached operation (16 MiB cap) for offline profiling; with the `flint-bench-hooks` feature, `SYMGCAD_CAPTURE_ALL` keeps every operation (used for Section 3). diagnostic
- R5 **Throttled progress reporting.** Progress files are written at a bounded rate. *Effect:* an unthrottled report had cost 17.505 s of system CPU in a 60 s C470 trial; now 1.282 s. default-on
- R6 **Buffered result export.** Large proofs are written through one buffered writer. *Effect:* an A286 run that failed through 392,791 tiny writes completes 3,308 cells in 1.354 s. default-on
- R7 **Interrupted-worker accounting.** Timed-out workers keep their elapsed time and typed diagnostics, so incomplete results stay explainable. default-on
- R8 **Bounded parsing of untrusted hint text.** Size, degree and coefficient bounds are checked on the parse tree before Symbolica expands anything. *Effect:* `(x+y+z+1)^300` was rejected after 6.9 s, now in < 0.5 s; `7^(3^20)`, which never returned, is rejected. NEW 2026-10-12 default-on
- R9 **Fully supervised portfolio and pieces.** Portfolio enumeration, probes and `pieces plan/assemble` run in bounded workers; any failure falls back to MODS with the reason recorded. *Effect:* a 3 s wall test stops at 3.08 s with 491–518 MB peak (before: 19.1 s, 1.97 GB). NEW 2026-10-12 default-on
- R10 **Reproducible environment.** Nix flake and Cargo lock with a pinned public Symbolica revision; no private checkout needed (symGCAD PR #1); CPU-affinity tools for pinned measurements. default-on
- R11 **Explicit five-hour budgets.** `wall_time_secs` and the campaign, solve and verification budgets accept up to 18,000 s (previously 7,200 s). The one-hour default and the memory, worker and cell limits are unchanged. A longer budget does not make a blocking native call interruptible or turn a partial result into a complete one. NEW 2026-10-12 opt-in (`[limits] wall_time_secs`; campaign `campaign_time_secs`, `solve_timeout_secs`, `verify_timeout_secs`)

2.11 Interfaces

- I1 **Rust library and TOML command line.** `solve`, `verify`, `project`, `order`, `bench`, `corpus`, `import-fastsecdec`. Inputs take exact rationals, `[parameters]`, `variable_groups`, binding orders and `[limits]`; unknown fields are refused (GCAD: a Python API on SymPy expressions). default-on
- I2 **`project` matches `solve`.** `symgcad project` now exports exactly the projection that `solve` lifts, including the homogeneous reduction, so projection sizes can be studied without solving. *Effect:* Gluza33 `R+-`: 354.5 s / 453 MB, before a timeout at 1,500 s. NEW 2026-10-12 default-on
- I3 **Piece-wise solving from the command line.** `symgcad pieces plan` writes one TOML per `bilinear_general` piece; pieces are solved as separate processes in any order or on any machine; `pieces assemble` refuses duplicate, missing or foreign results and joins them into one verifiable proof. NEW 2026-10-12 opt-in
- I4 **FastSecDec graph import.** Reads native signed `U/F`, exact kinematics and edge-to-parameter maps from dressed HEPKit DOT graphs and checks them before solving. opt-in (`import-fastsecdec`)
- I5 **Rust corpus builder.** Turns GCAD's 177 collected tests, 265 literal two-loop and 12 generated one-loop fixtures into 331 exactly deduplicated problems (124 fixed ZZ, 124 symbolic ZZ, 83 core). default-on
- I6 **Generated benchmark document.** This report: `benchmark/build_report.py` turns the cold symGCAD runs, the native GCAD runs, the long attempts and the FLINT measurements into `report-data.json` with per-row provenance; Typst renders it; `benchmark/run_final.sh` repeats the whole campaign with one command. NEW 2026-10-12 default-on
- I7 **Reproducible full-corpus campaign.** One manifest of the 331 groups and their recipes, a cold symGCAD runner (fresh run directory and `TMPDIR` per run, no persistent checkpoints except a fresh directory for MCTS searches, one pinned CPU and its own worker process per group, independent `verify`) and a native runner (unmodified GCAD through the unmodified bridge, 300 s per sign case, 8 GiB). Both are resumable and record binary hashes. NEW 2026-10-12 diagnostic
- I8 **Symbolica vs FLINT harness.** Exact operands captured from real solves are replayed one per fresh process through Symbolica's production kernels and FLINT 3.6, with canonical output comparison (Section 3, `benchmark/FLINT_vs_symbolica/`). The capture hooks sit behind the off-by-default cargo feature `flint-bench-hooks` (example `flint_vs_symbolica`). NEW 2026-10-12 diagnostic

2.12 Upstream dependency fixes

- U1 **Symbolica PR #69: resultant sign with a linear argument.** `Res(x^3+y, x+2)` returned `y-8` instead of `8-y` in every public strategy. symGCAD uses its own signed identity for linear operands; the patch is optional. optional patch
- U2 **Symbolica PR #75: integer left-shift overflow.** `Integer::from(1) << 63` gave a negative number; the patch is checked by 546 comparisons. No production path was affected. NEW 2026-10-12 (PR opened) optional patch
- U3 **Symbolica PR #76: Lazard–Ducos panic on a vanishing subresultant.** Explicit zero coefficients made Ducos index out of bounds; the patch comes with a regression test that fails before and passes after; production avoids the panic through A8. NEW 2026-10-12 optional patch
- U4 **Further upstream reports.** PRs #61 (determinant sign on odd row swaps), #67 and #68 (F4 Gröbner zero generators and cache), #73 (zero-variable exponent iteration), found while auditing Symbolica; no local patch is needed. upstream only
- U5 **Bounded small-leading-coefficient Wang factorization.** Extends Symbolica's Wang factorization to small non-monomial leading coefficients with at most 12 candidate tuples. *Effect:* 2,410-term Gluza33 factor 1.039 s, against a 60.03 s timeout without it. optional patch (experimental)
- U6 **Bounded main-axis choice for factorization.** Lets the factorizer pick the lowest-degree main variable within bounds. *Effect:* 1,731-term component 0.280 s against a 30.03 s timeout; the whole 2,867-term source 1.951 s. optional patch (experimental)
- U7 **Positive-only isolation and unit Taylor shift.** Research backend patches for Symbolica's real-root isolation: isolate only the positive roots when only those matter, and add a cheap exact path for the unit Taylor shift `p(x+1)` used by Descartes-based isolation. *Effect:* unit shift −39% on an authentic operand; positive-root-only isolation 557.98 s / 421 MiB against 765.81 s / 813 MiB for all roots. optional patch (research backend)
- U8 **Idempotent patch bootstrap.** `scripts/bootstrap-deps.sh --apply-patches` applies the optional fixes to a separate research checkout; the default build stays pristine. opt-in

2.13 Opt-in contract extensions

- X1 **Projective sections (opt-in contract extension).** With `solver.projective_sections = true` (engine `bilinear_general` only, default off), a cell's upper boundary may be "the k-th positive root of P, or +∞ where P has fewer roots": inside one cell the boundary can pass continuously through infinity. Cells stay connected and cylindrical after the map `u = x/(1+x)`; consumers that do not know this domain fail on the `"+inf"` sample strings instead of misreading them. For `F = a*x*y + b*x + c*y + d` with the replayed ratio premises and `b*c - a*d = m*G^2`, the fiber over a base point is empty when `d < 0` and exactly `{0 < y < Y*, 0 < x < X(y)}` when `d > 0`, so only `d > 0` has to be decomposed, one cell per base cell. The verifier replays premises and plan; the other verifiers, the merge and the cell comparison refuse or handle such bounds explicitly. Approved by the user on 2026-10-12. *Effect:* positive Gluza33 (unresolved under the strict contract, longest attempt 7,200 s): 2,372 verified cells, solve 0.38 s, verify 0.54 s in this report's cold run; a seeded sanity check of 4,744 fresh points (on an identical decomposition) evaluated the original `F` exactly with 0 failures. Positive Gluza30: 1 cell in 0.06 s instead of 80,980. Not counted in the strict totals; it does not apply to the sign-split symbolic problems. NEW 2026-10-12 contract extension opt-in
- X2 **Aggressive merging.** GCAD-compatible merging that may cross seams; only as a separate output (V5). opt-in

2.14 Investigated, not adopted

- N1 **Convex two-point theory (2026-10-12).** For two-point graphs, `F/U` is convex; dropping touching pairs shrinks the Gluza33 level-2 projection from 86 to 71 factors under the current cell contract, and to 34 only with "generalized sectors", a contract change that was not implemented. diagnostic
- N2 **Relevance-restricted projection prototype (2026-10-12).** No-go: about 1.9 million base and 3.0 million 8-variable cells for positive Gluza33 under the strict contract (ordinary CAD: about 10^{10} on one piece). diagnostic
- N3 **Literature and Symbolica audits (2026-10-12).** Landau, PLD and Whitney sets are necessary but not sufficient for cylindrical cells; Symbolica already supplies every primitive used, and over `Q(√10)` it splits Gluza33 discriminant factors (450 ⟶ 43 × 43 terms in 1.05 s) but stalls on 2,250 terms. diagnostic
- N4 **Domain-aware ordering profile (2026-10-12).** Anchors and orders picked by predicted kinematic cells made solved symbolic test beds 2–312× faster (led to O5); no effect on the unresolved two-loop cases. diagnostic
- N5 **Earlier physics-guided research lines.** CFF and Landau chambers, tropical and SubTropica ordering, graph-derived quadratic blocks, ordered quadratic flag identities and crossing transport produced exact identities or ordering proposals but no production gain; physical intuition never authorizes dropping a boundary. diagnostic

- N6 **Unresolved-case campaign (2026-10-12).** 24 of the 28 unresolved originals were re-run for one hour each with the improved local recipes (sibling retention, no pair witnesses); none completed. The rows of Section 1 show these attempts as lower bounds (tag ^{w}). diagnostic
- N7 **Earlier algebra research lines.** Deterministic CRT prime images, factorial-convolution Taylor shifts (correct, but 0.67 s against 0.27 s native on a degree-1,478 block), exact centering and discriminant-of-resultant reductions, three-parameter segmentation, triangular kinematic shifts, rational kinematic pair maps, quadratic-face projection and multivariate square-free product diagnostics: exact and documented, not promoted. diagnostic
- N8 **Local-engine proposals not implemented.** Moving-scale Descartes sections, full-real root covers from untrusted hints, constant lower-ray certificates, selected positive common roots without a full discriminant, positive-root remainder signs and two-coordinate homogeneous charts are written up as designs with their proof obligations (reference notes), but have no production dispatch. diagnostic

3 Symbolica vs FLINT on the algebraic steps

symGCAD runs all exact polynomial algebra through Symbolica; the original GCAD calls FLINT. To isolate the algebra from everything else, exact operands were captured from real symGCAD solves (Gluza30/33, A317, A5_31, fixed A377 and others), written in a neutral format, and replayed one operation per fresh process through (i) the kernel symGCAD's production code actually uses (Symbolica, after symGCAD's own dispatch: closed-form discriminants, interpolation, binary segmentation), (ii) Symbolica's individual algorithms (Lazard–Ducos subresultants, Brown/CRT), and (iii) FLINT 3.6 (`fmpz_mpoly` and `fmpz_poly`). Every output pair is compared after canonicalization (signs and content); 441 outputs were compared with 0 mismatches. Times are the minimum over repetitions on one pinned CPU; **ratio = Symbolica time / FLINT time**, so a ratio above 1 means FLINT is faster.

RESULTANT
0.67
geometric-mean ratio over 56 of 67 operands; total-time ratio 0.19

DISCRIMINANT
0.51
geometric-mean ratio over 28 of 34 operands; total-time ratio 0.22

FACTOR
2.28
geometric-mean ratio over 42 of 42 operands; total-time ratio 5.9

GCD
2.31
geometric-mean ratio over 24 of 24 operands; total-time ratio 9.32

Z[X] RESULTANT (SEGMENTATION)
0.96
geometric-mean ratio over 14 of 26 operands; total-time ratio 0.97

Summary. By geometric mean: resultant Symbolica faster (0.67); discriminant Symbolica faster (0.51); factor FLINT faster (2.28); gcd FLINT faster (2.31); Z[x] resultant (segmentation) about even (0.96). The geometric mean weights every operand equally, so it is dominated by small, sub-millisecond operations; the total-time ratio is dominated by the few largest operands. Resultants and discriminants dominate projection cost; factorizations and GCDs build its square-free bases. Best case for Symbolica: `symA317-dis-10-d9e5bd74c8` (ratio 0.06); best case for FLINT: `g33S2-fac-31-70f8238aed` (ratio 22.1).

Status of the measurements. The numbers are the committed snapshot in `benchmark/FLINT_vs_symbolica/results/` (2026-10-12, 12:57 UTC): 195 operands, 646 of 772 timing jobs run (39 stopped at their cap, mostly exploratory 120 s kernels), 126 jobs on 30 large-tier operands not yet measured. Those 30 operands (nine of the eleven A377 pair resultants, the four Gluza33 critique discriminants and the two interpolation images derived from them, the stalled Gluza33 main-axis resultant, two operations in progress at the harvest deadlines and the 12 large-tier segmentation encodings) are listed as *pending*, never as timeouts. Each job ran once; a 26-operand subset re-run twice from clean builds reproduced all 88 exact outputs and the times within 0.85–1.05×.

Interpretation.

- **Resultants and discriminants: symGCAD's production path beats FLINT on large inputs, by algorithm choice.** For the large pair resultants symGCAD packs two parameters into one integer variable (binary segmentation, item A9) and computes a single univariate integer resultant: 3.3–6.8× faster than FLINT's `fmpz_mpoly_resultant` on the segmented Gluza33 resultants (and 3.2–7.5× faster than `fmpz_mpoly_discriminant` on the segmented Gluza33 discriminants), 3.5× on A317, 10.5× on A318, 15.7× on one A377 pair, and 34 s on another A377 pair where FLINT did not finish in 600 s. Closed-form discriminants (A5) are 6–18× faster than `fmpz_mpoly_discriminant` on the larger parametric ones.
- **Like-for-like kernels are close.** Both libraries' Lazard–Ducos subresultants: Symbolica 0.74–0.88× FLINT's time on large operands, about 1.0–1.1× on small ones; the univariate integer resultant inside segmentation is at parity (0.93–1.07). In the subset re-run (the snapshot tables still list both interpolation images as pending), FLINT's multimodular univariate resultant was 1.7× faster on one interpolation image (10.0 ms against 16.9 ms); FLINT's default `fmpz_poly_resultant` dispatcher picks a multimodular method that does not finish on the segmented operands.
- **Factorization and GCD: FLINT is 3–22× faster on large dense bivariate inputs.** On the dense Gluza33 factor operands (13–21 k terms) FLINT's `fmpz_mpoly_factor` takes 0.17–2.8 s against Symbolica's 3.2–10.6 s, and gcd(P, ∂P) 0.14–0.27 s against 1.3–5.0 s; FLINT's dense Brown-type modular GCD suits these shapes. On the sparse 4–5-variable factorizations and GCDs of the parametric problems Symbolica is mostly faster (0.17–0.77; GCDs 0.19–1.9). This is the most valuable upstream target for Symbolica.
- **What it means for symGCAD.** Projection cost in the hard cases is dominated by large resultants and discriminants, where the production path is already faster than FLINT; the remaining kernel gap is in dense multivariate GCD and factorization. No output differed between the libraries.

Reproduction. Harness (Symbolica and FLINT sides), operand manifest with sha256 and provenance, nix build/run commands, the subset re-run and six optimization ideas: `benchmark/FLINT_vs_symbolica/README.md`. The Symbolica side is built with `cargo build --release --locked --features flint-bench-hooks --example flint_vs_symbolica`; the feature is off by default.

FLINT 3.6.0 vs Symbolica (symGCAD production kernels): 195 operands, 646 of 772 timing jobs run (snapshot 2026-10-12 12:57 UTC, campaign still running); 441 exact outputs compared, 0 mismatches. Jobs stopped at their cap: 39 (3 at 600 s, 2 at 300 s, 34 at 120 s; only FLINT and Symbolica Ducos on a377-1d3e6e9835cc are non-exploratory); not yet run: 126.

Operation	n	both timed	capped	not run	geo. mean ratio	total-time ratio
resultant	67	56	1	10	0.67	0.19
discriminant	34	28	0	6	0.51	0.22
factor	42	42	0	0	2.28	5.90
gcd	24	24	0	0	2.31	9.32
Z[x] resultant (interp. image)	2	0	0	2	–	–
Z[x] resultant (segmentation)	26	14	0	12	0.96	0.97

Table 1: Symbolica (symGCAD production path) / FLINT 3.6.0 time ratio by operation type. Total-time ratio = sum of Symbolica times / sum of FLINT times over operands timed on both sides.

Operation	Size class	n	**Production / FLINT** (geo. mean)	Range	Symbolica faster	Ducos / FLINT mpoly
resultant	<1 ms	32	1.08	0.38–2.74	16/32	1.08 (29)
resultant	1-100 ms	5	0.84	0.37–1.73	3/5	0.84 (5)
resultant	0.1-10 s	3	0.98	0.94–1.02	2/3	0.98 (3)
resultant	>=10 s	16	0.23	0.06–0.80	16/16	0.79 (15)
resultant	capped	1	–	–	0/0	–
discriminant	<1 ms	18	0.76	0.30–1.71	13/18	1.13 (10)
discriminant	1-100 ms	4	0.31	0.06–1.24	3/4	0.94 (4)
discriminant	0.1-10 s	1	0.06	0.06–0.06	1/1	1.54 (1)
discriminant	>=10 s	5	0.29	0.13–0.83	5/5	0.78 (5)
factor	<1 ms	17	1.03	0.17–9.84	10/17	–
factor	1-100 ms	17	2.66	0.45–14.2	4/17	–
factor	0.1-10 s	7	8.52	2.66–22.1	0/7	–
factor	>=10 s	1	11.7	11.7–11.7	0/1	–
gcd	<1 ms	10	1.21	0.19–3.00	3/10	–
gcd	1-100 ms	6	1.15	0.39–3.42	3/6	–
gcd	0.1-10 s	8	8.70	5.72–18.4	0/8	–
Z[x] resultant (segmentation)	0.1-10 s	1	0.94	0.94–0.94	1/1	1.68 (1)
Z[x] resultant (segmentation)	>=10 s	13	0.96	0.93–1.07	12/13	1.63 (13)

Table 2: Time ratios by operation type and size class (class of the slower side; capped = a side hit its cap). Last column: Symbolica integer Ducos (production's deterministic fallback) / fmpz_mpoly.

Operand	Active vars; degrees	Terms	Production kernel	Symbolica production	Best FLINT	Ratio	Symbolica Ducos
a317-pair	s,t,x0; (10,6,12); (10,6,12)	324+323	binary-segmentation	29.2 s	101 s	0.29	91.0 s
a318-pair	s,t,x1; (22,20,10); (8,8,6)	2560+161	binary-segmentation	15.7 s	165 s	0.10	144 s
a377-1d3e6e9835cc	s,t,x0; (16,6,12); (13,6,8)	1009+575	binary-segmentation	34.1 s	> 600 s	–	> 300 s
a377-1d4048e35d3e	s,t,x0; (14,7,8); (16,8,8)	783+885	binary-segmentation	14.6 s	230 s	0.06	not run
g33S2-res-00-11a890ffaa	x1,x2,x7; (8,12,12); (12,12,12)	679+993	binary-segmentation	36.7 s	201 s	0.18	158 s
g33S2-res-01-92460725ec	x1,x2,x7; (10,10,12); (12,12,12)	705+993	binary-segmentation	25.4 s	160 s	0.16	124 s
g33S2-res-02-41f64e7684	x1,x2,x7; (12,8,12); (12,12,12)	679+993	binary-segmentation	17.3 s	118 s	0.15	91.6 s
g33S2-res-03-f3f5c3be5d	x1,x2,x7; (12,12,12); (10,10,11)	993+630	binary-segmentation	21.3 s	142 s	0.15	111 s
g33S2-res-04-1b1af7b175	x1,x2,x7; (8,12,12); (10,10,12)	679+705	binary-segmentation	20.0 s	91.0 s	0.22	71.8 s
g33S2-res-05-83bfdd39e0	x1,x2,x7; (8,10,10); (12,12,12)	457+993	binary-segmentation	18.3 s	88.9 s	0.21	69.6 s
g33S2-res-06-e4dacfd3fb	x1,x2,x7; (8,12,12); (10,10,11)	679+630	binary-segmentation	17.1 s	81.9 s	0.21	63.5 s
g33S2-res-07-1c09fee64d	x1,x2,x7; (8,12,12); (12,8,12)	679+679	binary-segmentation	14.6 s	77.1 s	0.19	60.0 s
g33S2-res-08-17eaa2af35	x1,x2,x7; (8,12,12); (8,10,10)	679+457	binary-segmentation	14.1 s	46.9 s	0.30	37.2 s
g33S2-res-09-6ab3116dc1	x1,x2,x7; (10,10,12); (10,10,11)	705+630	binary-segmentation	12.1 s	59.5 s	0.20	45.9 s
g33S2-res-10-213a6a967e	x1,x2,x7; (10,4,10); (12,12,12)	294+993	integer-ducos	11.8 s	14.7 s	0.80	11.7 s
symA317-res-00-c0eed4af4e	s,t,x0,x2; (6,4,4,4); (9,6,8,8)	229+1307	integer-ducos	34.7 s	47.0 s	0.74	34.9 s
symA317-res-01-6bbe449f39	s,t,x0,x2; (4,2,4,4); (9,6,8,8)	126+1307	integer-ducos	16.3 s	23.4 s	0.70	16.1 s
g33S2-dis-20-569a4bfe48	x1,x2,x7; (12,12,12)	993	binary-segmentation	37.2 s	278 s	0.13	206 s
g33S2-dis-21-d3fad47e59	x1,x2,x7; (8,12,12)	679	binary-segmentation	21.5 s	68.1 s	0.32	51.2 s
g33S2-dis-22-60a726bccc	x1,x2,x7; (12,8,12)	679	integer-ducos	17.6 s	21.2 s	0.83	18.7 s
g33S2-dis-23-8acb497b1c	x1,x2,x7; (10,10,12)	705	binary-segmentation	13.1 s	48.5 s	0.27	36.8 s
g33S2-dis-24-398386d3e8	x1,x2,x7; (10,10,11)	630	binary-segmentation	7.6 s	34.8 s	0.22	26.7 s
g33S2-fac-28-3a75cafd52	x1,x7; (228,202)	21469	symgcad-factor-basis	10.6 s	905.8 ms	11.7	–
g33S2-fac-29-741a8fb652	x1,x7; (208,186)	18643	symgcad-factor-basis	7.4 s	2.8 s	2.66	–
g33S2-fac-30-3c86713b00	x1,x7; (184,168)	14995	symgcad-factor-basis	5.7 s	1.3 s	4.41	–
g33S2-fac-31-70f8238aed	x1,x7; (208,180)	16461	symgcad-factor-basis	5.5 s	247.0 ms	22.1	–
g33S2-fac-32-cc03af2bf8	x1,x7; (208,184)	17169	symgcad-factor-basis	5.4 s	1.7 s	3.11	–
g33S2-fac-33-d966d83ef5	x1,x7; (206,182)	18211	symgcad-factor-basis	4.0 s	310.9 ms	12.7	–
g33S2-fac-34-5056e9cf6a	x1,x7; (184,172)	14805	symgcad-factor-basis	3.5 s	214.9 ms	16.5	–
g33S2-fac-35-b4319ee36c	x1,x7; (168,160)	12905	symgcad-factor-basis	3.2 s	169.2 ms	19.2	–
g33S2-fac-28-3a75cafd52-gcd	x1,x7; (228,202); (227,202)	21469+21469	symbolica-gcd	5.0 s	269.3 ms	18.4	–
g33S2-fac-29-741a8fb652-gcd	x1,x7; (208,186); (207,186)	18643+18643	symbolica-gcd	1.8 s	256.2 ms	7.08	–
g33S2-fac-30-3c86713b00-gcd	x1,x7; (184,168); (183,168)	14995+14995	symbolica-gcd	1.3 s	191.9 ms	6.75	–
g33S2-fac-31-70f8238aed-gcd	x1,x7; (208,180); (207,180)	16461+16461	symbolica-gcd	2.4 s	203.7 ms	11.8	–
g33S2-fac-32-cc03af2bf8-gcd	x1,x7; (208,184); (207,184)	17169+17169	symbolica-gcd	1.5 s	257.0 ms	5.96	–

Operand	Active vars; degrees	Terms	Production kernel	Symbolica production	Best FLINT	Ratio	Symbolica Ducos
g33S2-fac-33-d966d83ef5-gcd	x1,x7; (206,182); (205,182)	18211+18211	symbolica-gcd	1.4 s	251.5 ms	5.72	–
g33S2-fac-34-5056e9cf6a-gcd	x1,x7; (184,172); (183,172)	14805+14805	symbolica-gcd	1.5 s	167.7 ms	9.17	–
g33S2-fac-35-b4319ee36c-gcd	x1,x7; (168,160); (167,160)	12905+12905	symbolica-gcd	1.4 s	140.5 ms	10.1	–
g33S2-dis-20-569a4bfe48-segenc	X; (12); (11)	13+12	integer-ducos-univariate	36.0 s	36.8 s (polyeuclid)	0.98	62.5 s
g33S2-dis-21-d3fad47e59-segenc	X; (12); (11)	13+12	integer-ducos-univariate	21.0 s	21.7 s (polyeuclid)	0.97	47.1 s
g33S2-dis-23-8acb497b1c-segenc	X; (10); (9)	11+10	integer-ducos-univariate	10.2 s	10.8 s	0.94	18.1 s
g33S2-dis-24-398386d3e8-segenc	X; (10); (9)	11+10	integer-ducos-univariate	7.3 s	7.7 s	0.94	13.0 s
g33S2-res-00-11a890ffaa-segenc	X; (12); (12)	13+13	integer-ducos-univariate	35.5 s	36.6 s (polyeuclid)	0.97	62.2 s
g33S2-res-01-92460725ec-segenc	X; (10); (12)	11+13	integer-ducos-univariate	24.2 s	25.9 s (polyeuclid)	0.93	42.2 s
g33S2-res-02-41f64e7684-segenc	X; (8); (12)	9+13	integer-ducos-univariate	19.4 s	18.1 s (polyeuclid)	1.07	28.4 s
g33S2-res-03-f3f5c3be5d-segenc	X; (12); (10)	13+11	integer-ducos-univariate	20.6 s	21.3 s (polyeuclid)	0.97	36.2 s
g33S2-res-04-1b1af7b175-segenc	X; (12); (10)	13+11	integer-ducos-univariate	19.5 s	20.6 s (polyeuclid)	0.95	34.8 s
g33S2-res-05-83bfdd39e0-segenc	X; (10); (12)	11+13	integer-ducos-univariate	17.6 s	18.2 s (polyeuclid)	0.97	30.3 s
g33S2-res-06-e4dacfd3fb-segenc	X; (12); (10)	13+11	integer-ducos-univariate	16.7 s	17.6 s (polyeuclid)	0.94	29.3 s
g33S2-res-07-1c09fee64d-segenc	X; (12); (8)	13+9	integer-ducos-univariate	14.2 s	14.9 s (polyeuclid)	0.95	24.9 s
g33S2-res-08-17eaa2af35-segenc	X; (12); (10)	13+11	integer-ducos-univariate	13.6 s	14.3 s (polyeuclid)	0.95	24.1 s
g33S2-res-09-6ab3116dc1-segenc	X; (10); (10)	11+11	integer-ducos-univariate	11.6 s	12.1 s (polyeuclid)	0.95	20.7 s

Table 3: Operands where either side needs at least 1 s or hits its cap (caps: 600 s production and FLINT, 300 s Symbolica Ducos, 120 s exploratory kernels).